

기초 영상처리

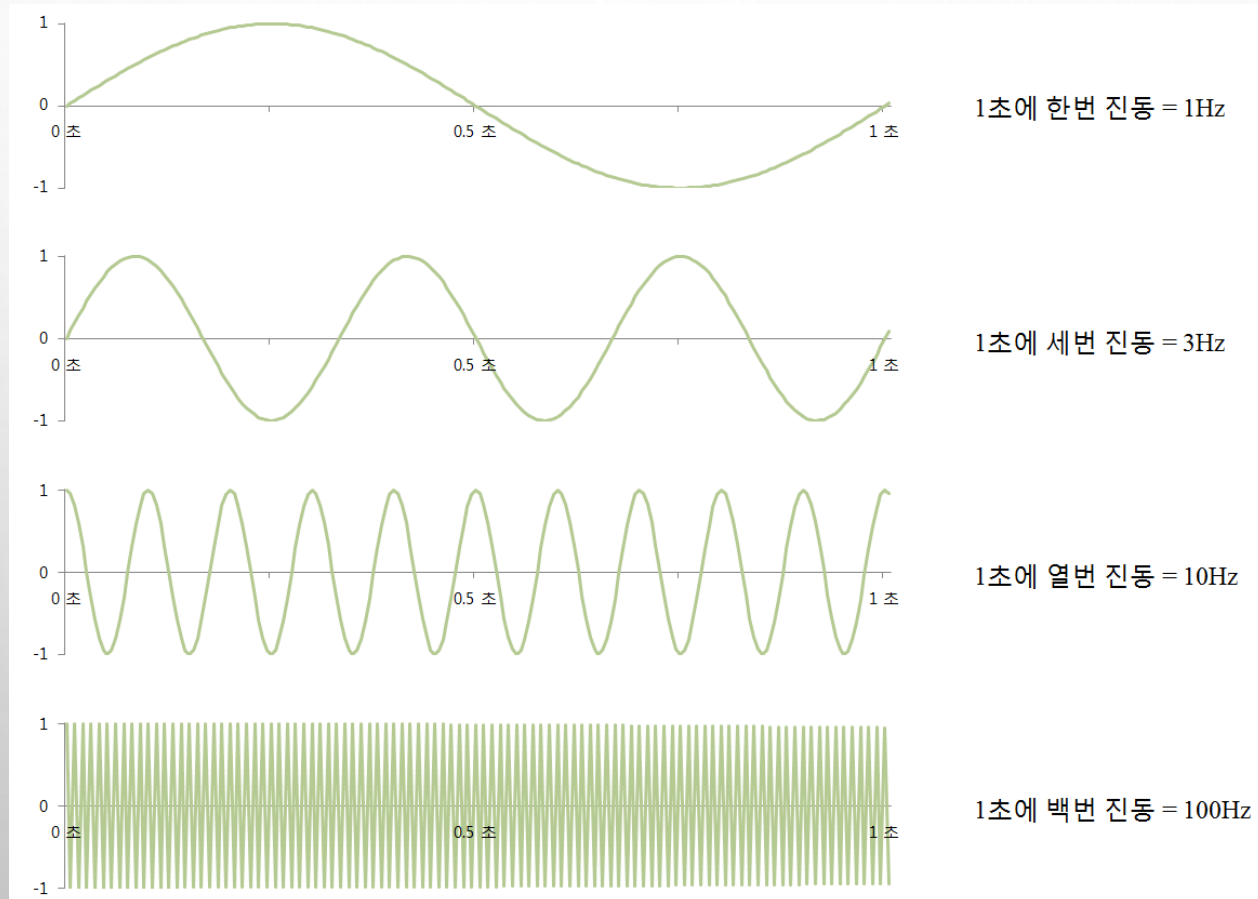
변환영역처리

9.1 공간 주파수의 이해

9.1 공간 주파수의 이해

◆ 헤르츠(Hz)

- ❖ 주파수를 표현하는 단위, 1초 동안에 진동하는 횟수
- ❖ 전파라는 신호에 국한된 표현

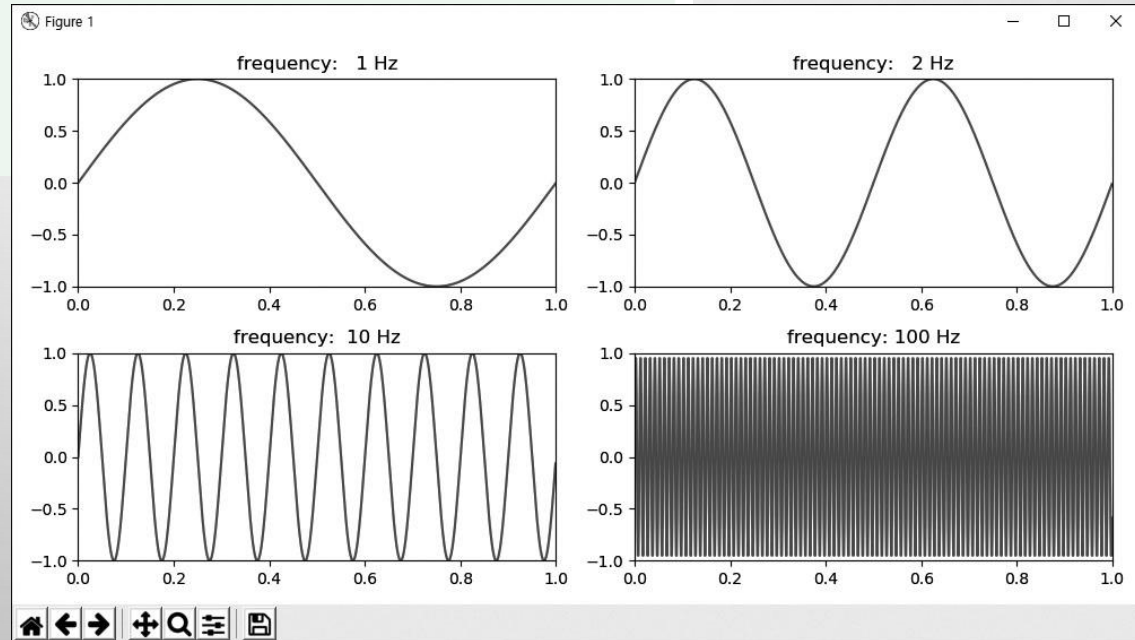


9.1 공간 주파수의 이해

예제 9.1.1

주파수 그리기 | - 01.frequency.py

```
01 import matplotlib.pyplot as plt          # 그래프그리기 라이브러리 임포트
02 import numpy as np
03
04 t = np.arange(0, 1, 0.001)                # 샘플링 범위 및 개수
05 Hz = [1, 2, 10, 100]                     # 주파수 예시
06 gs = [np.sin(2 * np.pi * t * h) for h in Hz]  # sin 함수 계산
07
08 plt.figure(figsize=(10, 5))
09 for i, g in enumerate(gs):
10     plt.subplot(2, 2, i+1), plt.plot(t, g)    # 그래프 그리기
11     plt.xlim(0, 1), plt.ylim(-1, 1)          # x, y축 범위 지정
12     plt.title("frequency: %3d Hz" % Hz[i])
13 plt.tight_layout()
14 plt.show()
```

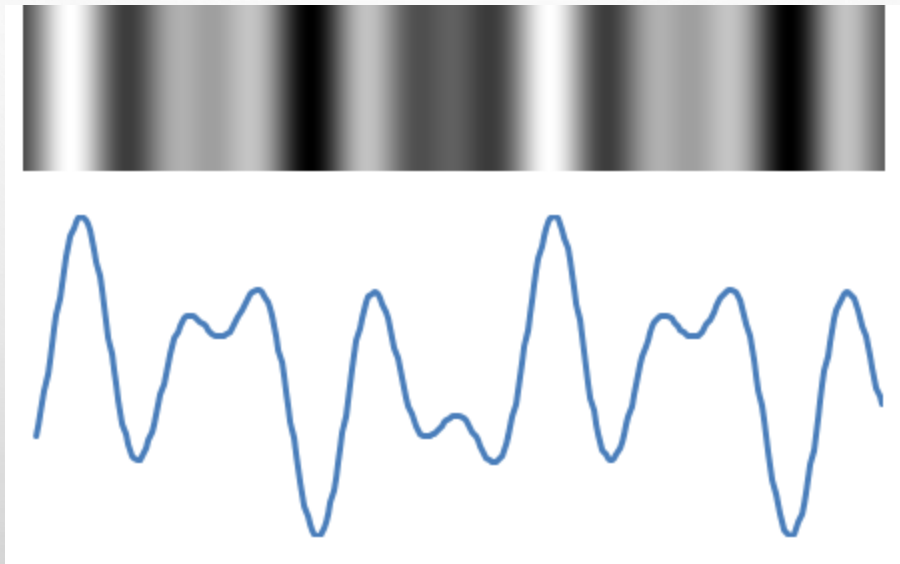


9.1 공간 주파수의 이해

◆ 영상처리에서는 공간 주파수(spatial frequency) 개념 사용

◆ 확장된 의미에서 주파수

- ❖ 이벤트가 주기적으로 재발생하는 빈도
 - 예) 화소 밝기의 변화도



9.1 공간 주파수의 이해

◆공간 주파수 - 밝기의 변화 정도에 따라서 고주파 영역/ 저주파 영역으로 분류

❖ 저주파 공간 영역

- 화소 밝기가 거의 변화가 없거나 점진적으로 변화하는 것
- 영상에서 배경 부분이나 객체의 내부에 많이 있음

❖ 고주파 공간 영역

- 화소 밝기가 급변하는 것
- 경계부분이나 객체의 모서리 부분



저주파 공간 영역

고주파 공간 영역

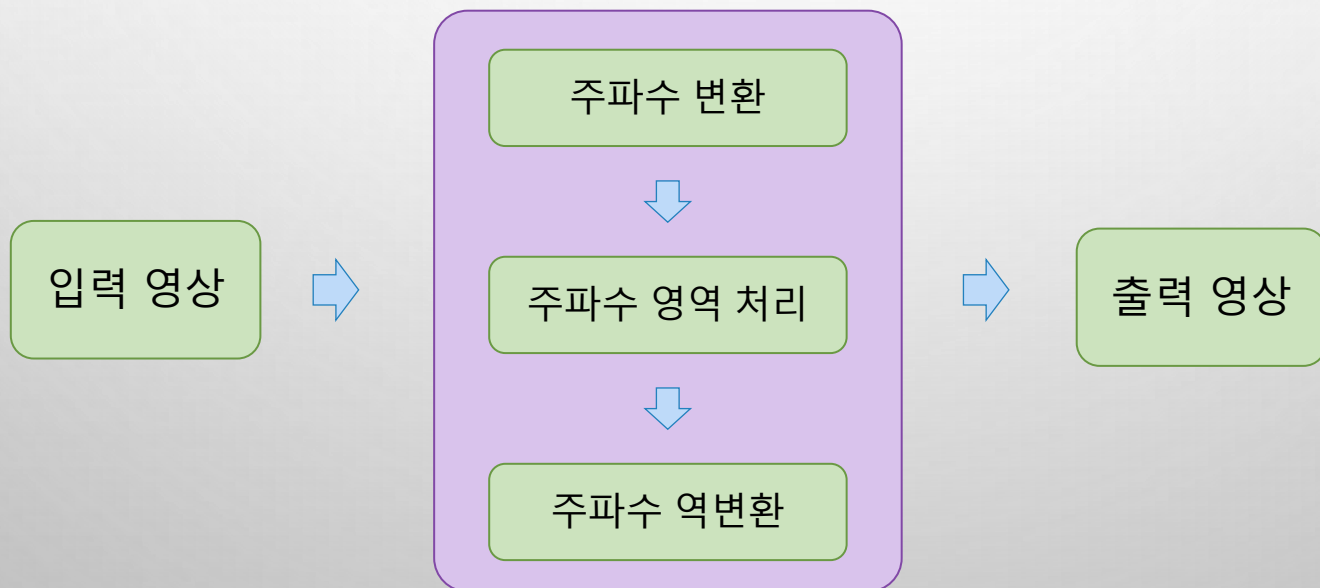
9.1 공간 주파수의 이해

◆영상을 주파수 영역별로 분리 한다면 ?

- ❖ 경계부분에 많은 고주파 성분 제거한 영상 → 경계 흐려진 영상
- ❖ 고주파 성분만 취한 영상 → 경계나 모서리만 포함하는 영상 즉, 에지 영상

◆공간 영역상에서 저주파 성분과 고주파 성분이 혼합 → 영역 분리해서 선별적 처리 어려움 → 변환영역 처리 필요

◆변환 영역 처리 과정



9.2 이산 푸리에 변환

◆ 시간축(공간축) vs 주파수영역

- ❖ 서로 다른 관점(축)을 기반으로 데이터를 분석

◆ 달리기 vs 자동차

- ❖ 달리기 : 100미터를 몇 초에?

- Time/Distance

- ❖ 자동차 : 1시간에 얼마를? (시속 얼마?)

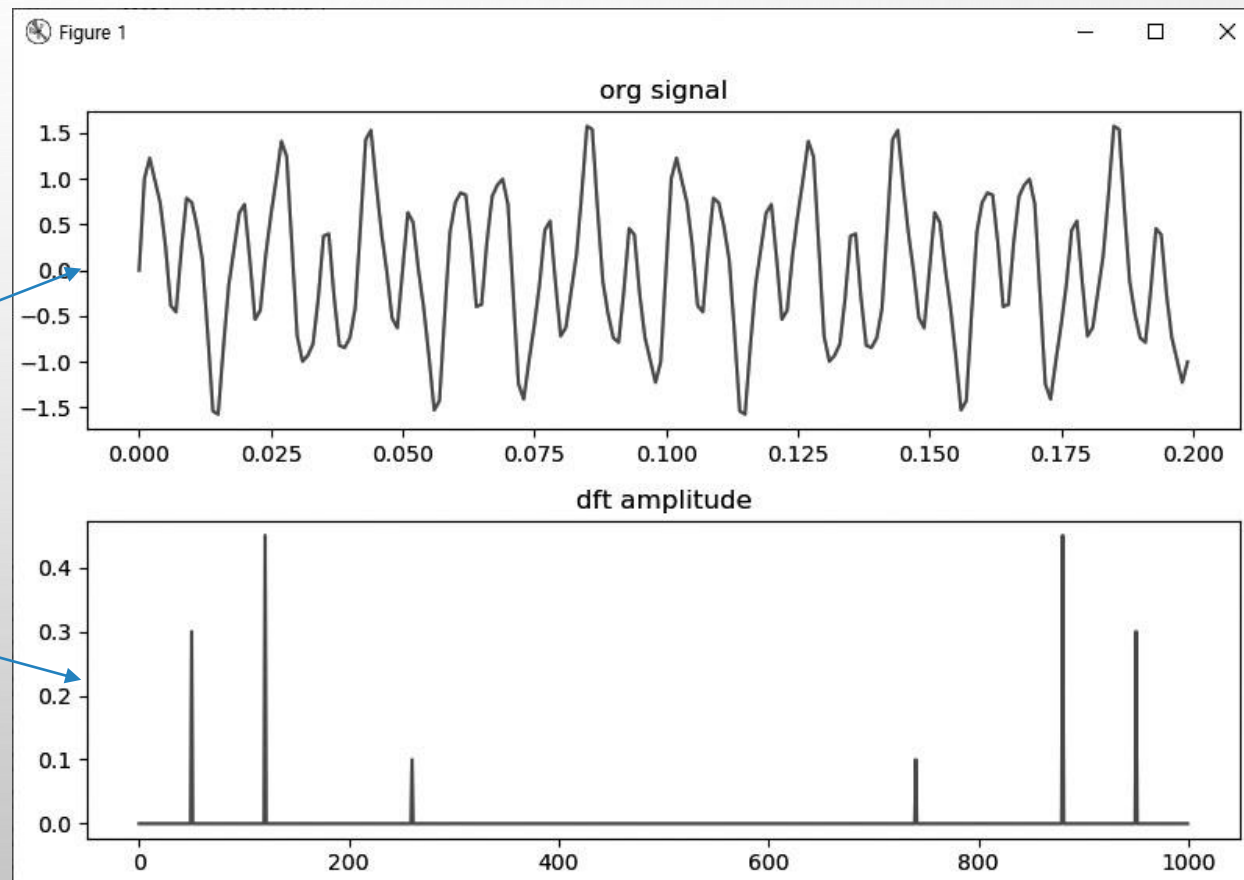
- Distance/Time

- ❖ 시간축 데이터

- Time/Cycle : Periods(주기/ sec) 1 번 돌 때 몇 초?

- ❖ 주파수 영역 데이터

- Cycle/Time(1 sec) = 진동수(Hz) 1 초동안 몇번?



9.2 이산 푸리에 변환

◆ 푸리에 변환을 이해하기 위해..

❖ 푸리에 급수(Fourier Series)

- 여러 개의 $\cos$ 과 $\sin$ 으로 어떤 것들이 표현이 가능한지

❖ 오일러 공식(Euler Equation)

- $\cos$ 과 $\sin$ 을 하나로 표현

❖ 적분(Integration)

- 적분의 의미

❖ 내적과 직교성(Orthogonality)

- 함수의 내적(Cross Product)

9.2 이산 푸리에 변환

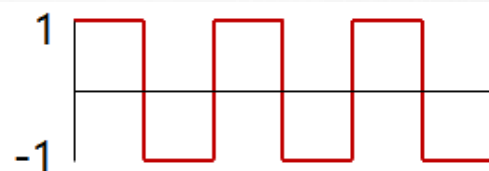
◆ 푸리에 변환

❖ 푸리에 급수(Fourier Series)

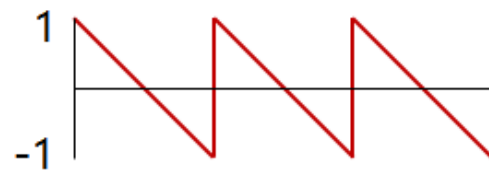
- 주기성을 갖는 신호는 $\sin$ (정현파)과 $\cos$ (여현파)의 조합으로 모두 표현이 가능하다.

$$s_N(x) = \underbrace{\frac{a_0}{2}}_{\text{DC(직류)}} + \underbrace{\sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi}{P}nx\right) + b_n \sin\left(\frac{2\pi}{P}nx\right) \right)}_{\text{AC(교류)}}$$

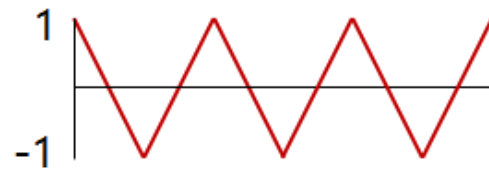
https://javalab.org/en/fourier_series_en/



$$f(x) = \frac{4}{\pi} \sum_{n=1,3,5,\dots}^{\infty} \frac{1}{n} \sin(nx)$$



$$f(x) = \frac{2}{\pi} \sum_{n=1,2,3,\dots}^{\infty} \frac{1}{n} \sin(nx)$$



$$f(x) = \frac{8}{\pi^2} \sum_{n=1,3,5,\dots}^{\infty} \frac{1}{n^2} \cos(nx)$$

9.2 이산 푸리에 변환

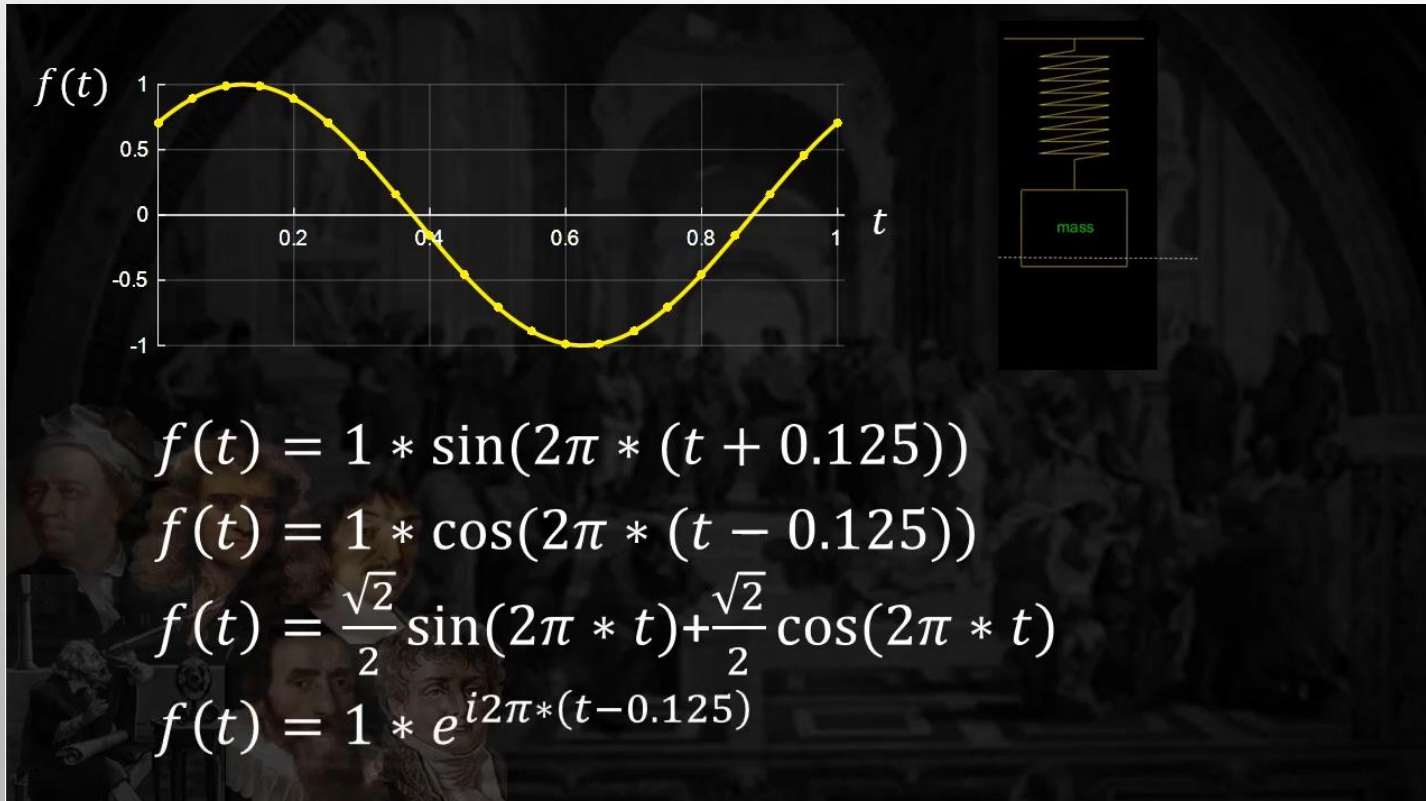
◆ 푸리에 변환

❖ 오일러 공식(Euler Equation)

- $\cos$ 과 $\sin$ 의 합을 하나의 수식으로 표현이 가능하도록 함

$$e^{ix} = \cos x + i \sin x$$

- (교재에서는 x 를 $w * t$ 로 사용하였음 : 시간 * cycle 개념으로 표현을 하기 위해서)



출처

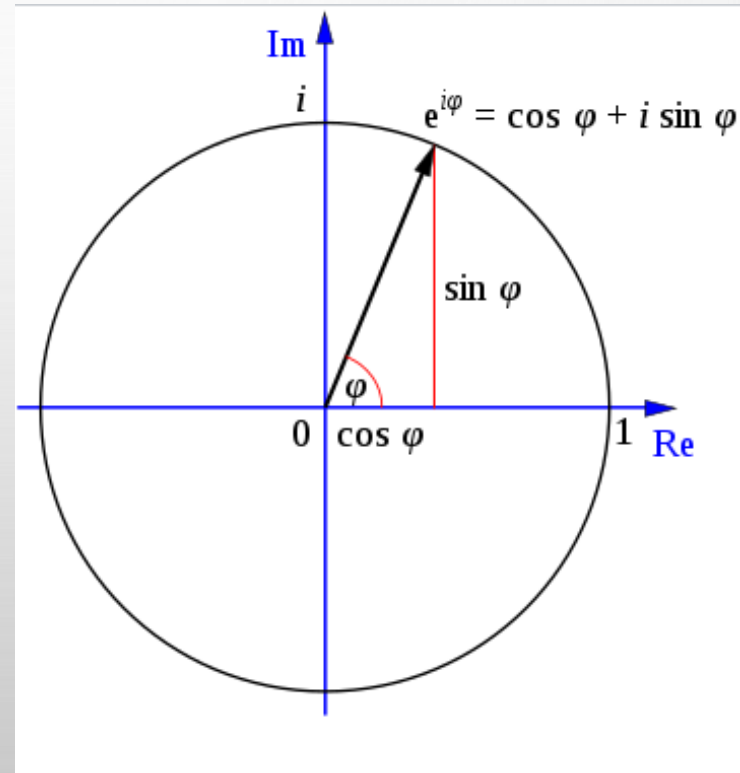
: <https://www.youtube.com/watch?v=8BuK47tbUKg>

9.2 이산 푸리에 변환

◆ 푸리에 변환

❖ 오일러 공식(Euler Equation)

- 좋은 이유는?
 - ✓ 지수 함수의 사용성 덕분
 - 적분/미분을 해도 똑같이 생김(적분의 경우 상수를 더해야 되지만..)
 - ✓ 두개의 벡터를 이용해서, 표현해야 하는 주기성 함수를
 - 수식 하나로 표현이 가능
 - 크기와 위상(각도)

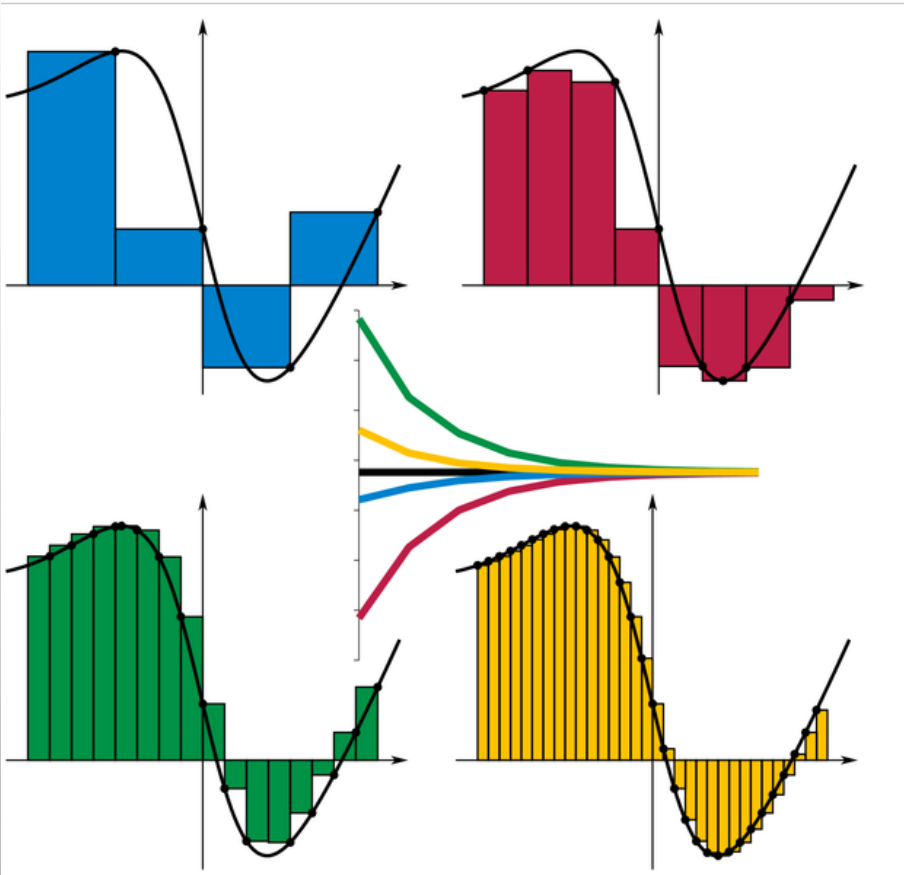


9.2 이산 푸리에 변환

◆ 푸리에 변환

❖ 적분

- 함수의 면적을 구하는 것



Riemann Integral

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i) \Delta x \quad \Delta x = \frac{b-a}{n} \quad \text{and} \quad x_i = a + i\Delta x$$

9.2 이산 푸리에 변환

◆ 푸리에 변환

❖ 내적을 구한다

- 연관성(Correlation) 을 찾는다.
- 벡터 내적

- 함수 내적

	벡 터	함 수
내적	$\vec{A} \cdot \vec{B} = \sum_{i=1}^n A_i B_i$	$\langle A B \rangle = \int_a^b A^*(x) B(x) dx$
크기	$ \vec{A} = \sqrt{\sum_{i=1}^n A_i^2}$	$ A(x) = \sqrt{\langle A A \rangle} = \sqrt{\int_a^b A^*(x) A(x)}$
직교 조건	$\vec{A} \cdot \vec{B} = 0$	$\langle A B \rangle = 0$

❖ 직교성

- 내적이 0일때 (서로 연관이 없다.)
 - ✓ 벡터 - 수직 관계
 - ✓ cos과 sin 간의 관계

9.2 이산 푸리에 변환

◆신호의 기본 전제

주기를 가진 신호는 정현파/여현파의 합으로 표현할 수 있다.

◆정현파/여현파

❖ 모든 파형 중에 가장 순수한 파형을 말하는 것으로 사인 또는 코사인 함수로 된 신호

◆주기를 갖는 신호

$$g(t) = 0.3 * g_1(t) - 0.7 * g_2(t) + 0.5 * g_3(t)$$

❖ 여러 사인 및 코사인 함수들로 분리 가능

● 분리된 신호(기저 함수):

$$g_1(t) = \sin(2\pi * t),$$

$$g_2(t) = \sin(2\pi * 3t),$$

$$g_3(t) = \sin(2\pi * 5t)$$

● 기저 함수에 곱해지는 값(주파수 계수): 0.3, -0.7, 0.5

9.2 이산 푸리에 변환

◆ 주파수 변환 수식

$$g(t) = \int_{-\infty}^{\infty} G(f) \cdot g_f(t) df$$

$g_f(t)$: f 주파수에 대한 기저함수
 $G(f)$: 기저함수의 계수

- 모든 주파수에 대한 기저함수와 그 계수들의 선형 조합
- ❖ 연속신호의 주파수 영역 변환
 - 존재하는 모든 기저함수에 대해서 그 계수인 $G(f)$ 를 구하는 것

◆ 푸리에 변환

- ❖ 사인이나 코사인 함수를 기저함수로 사용

$$g_f(t) = \cos(2\pi ft) + j \cdot \sin(2\pi ft) = e^{j2\pi ft}$$

허수 이용

- ❖ 푸리에 변환 – 원본 신호로부터 계수 $G(f)$ 를 얻는 것

$$G(f) = \int_{-\infty}^{\infty} g(t) \cdot e^{-j2\pi ft} dt$$

기저함수로부터 원본 신호를 만드는 것 → 푸리에 역변환

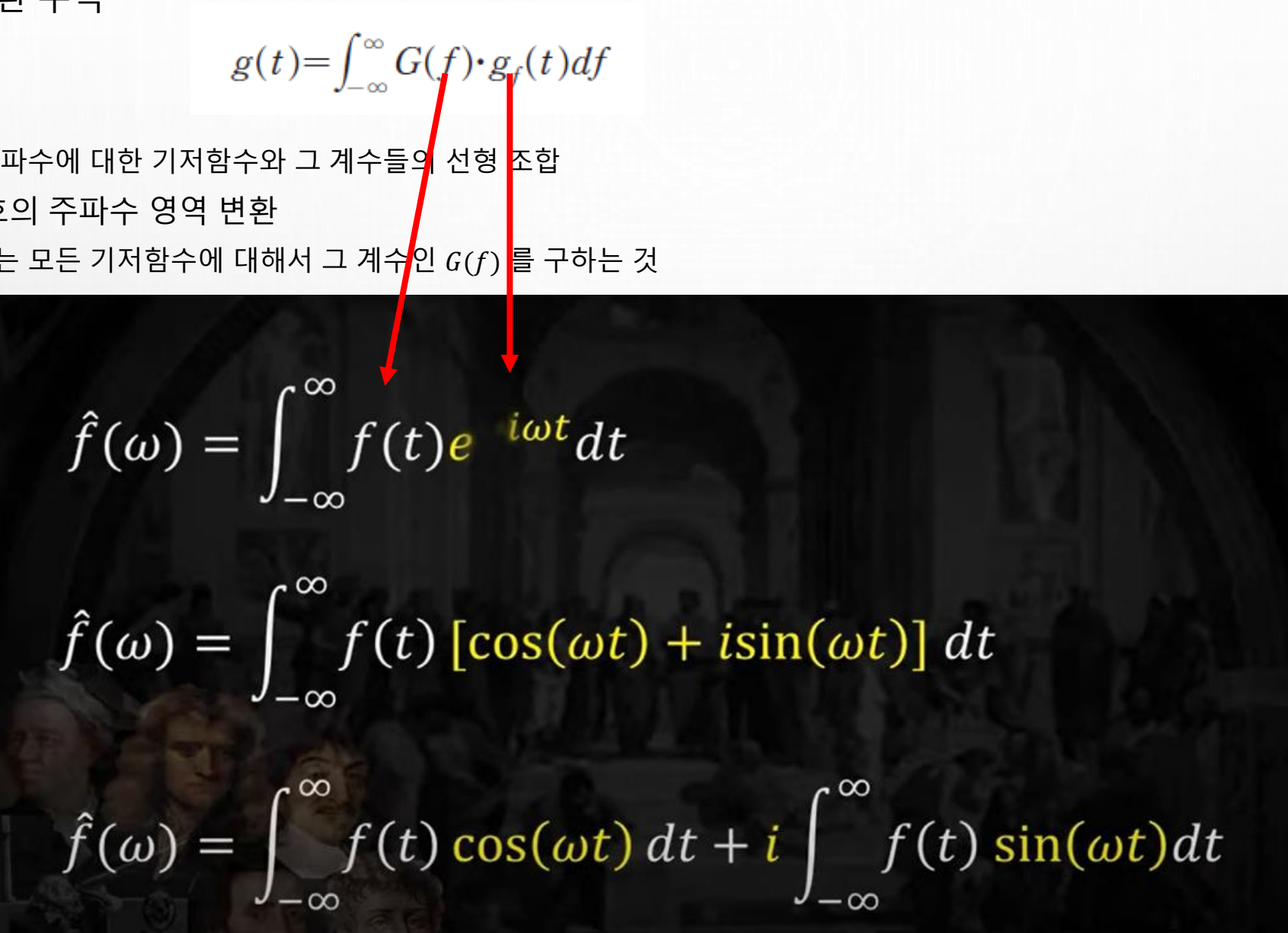
$$g(t) = \int_{-\infty}^{\infty} G(f) \cdot e^{j2\pi ft} df$$

9.2 이산 푸리에 변환

◆ 주파수 변환 수식

$$g(t) = \int_{-\infty}^{\infty} G(f) \cdot g_f(t) df$$

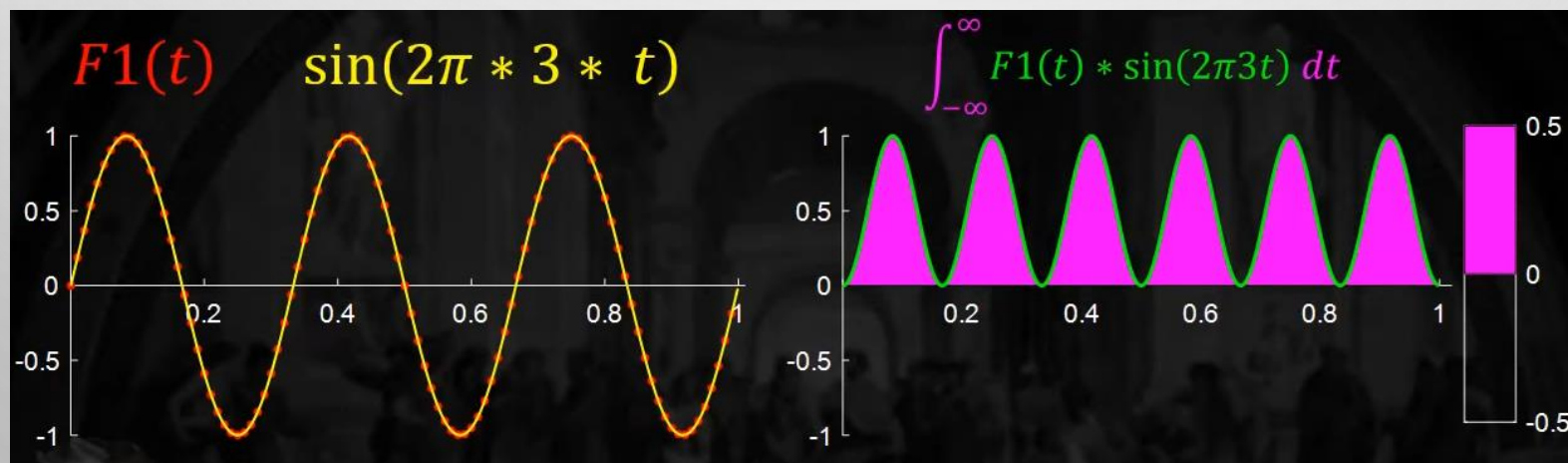
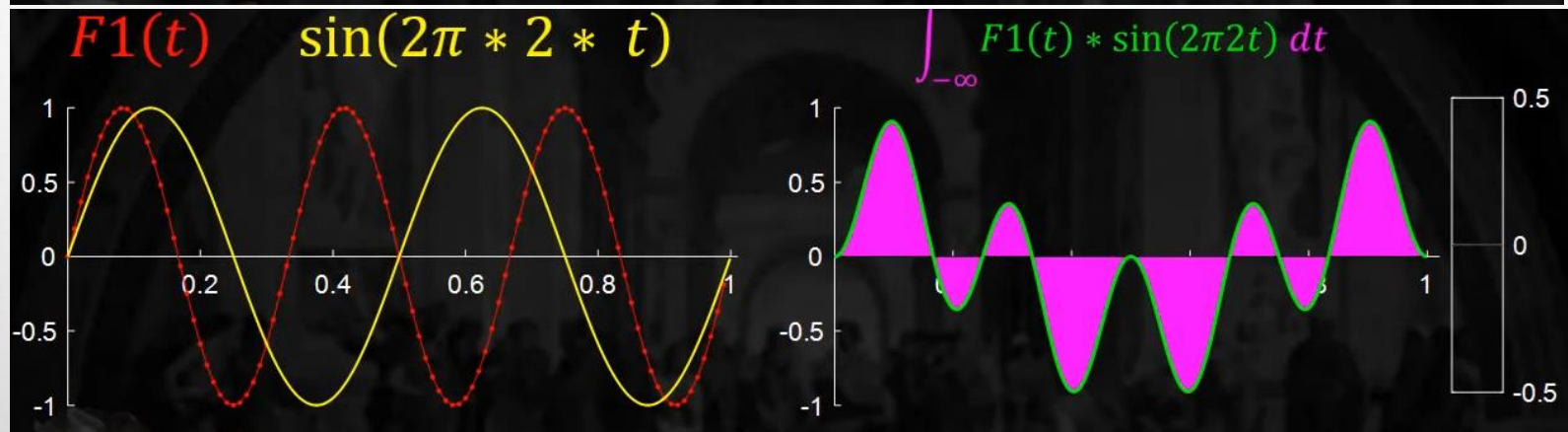
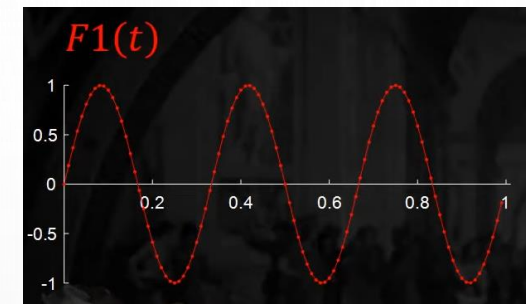
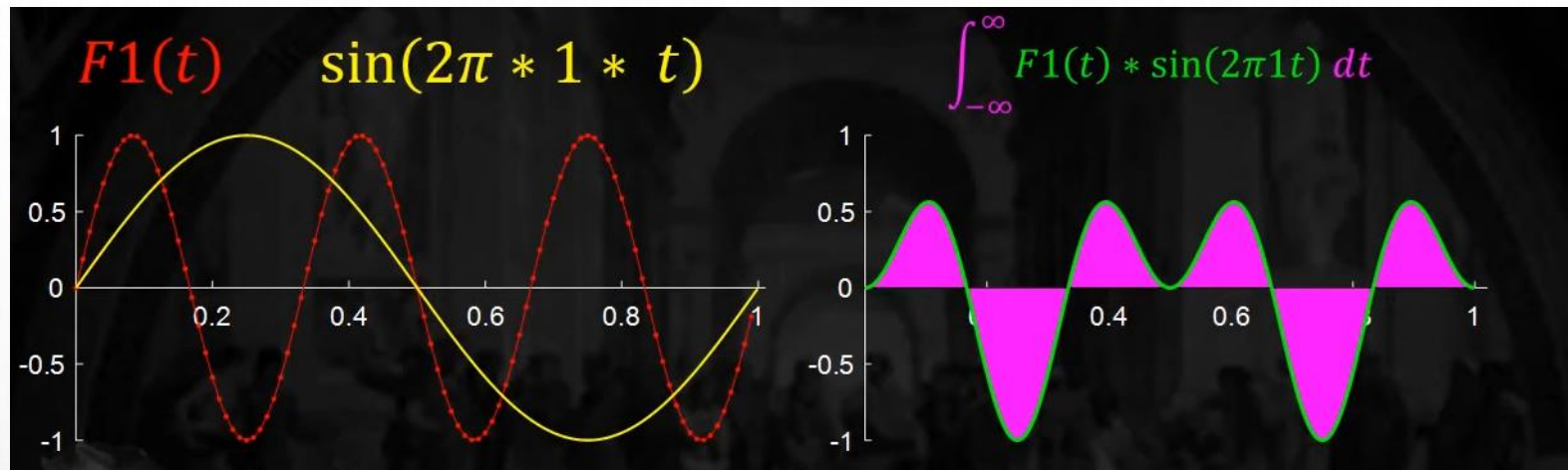
- 모든 주파수에 대한 기저함수와 그 계수들의 선형 조합
- ❖ 연속신호의 주파수 영역 변환
 - 존재하는 모든 기저함수에 대해서 그 계수인 $G(f)$ 를 구하는 것


$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) e^{i\omega t} dt$$

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) [\cos(\omega t) + i \sin(\omega t)] dt$$

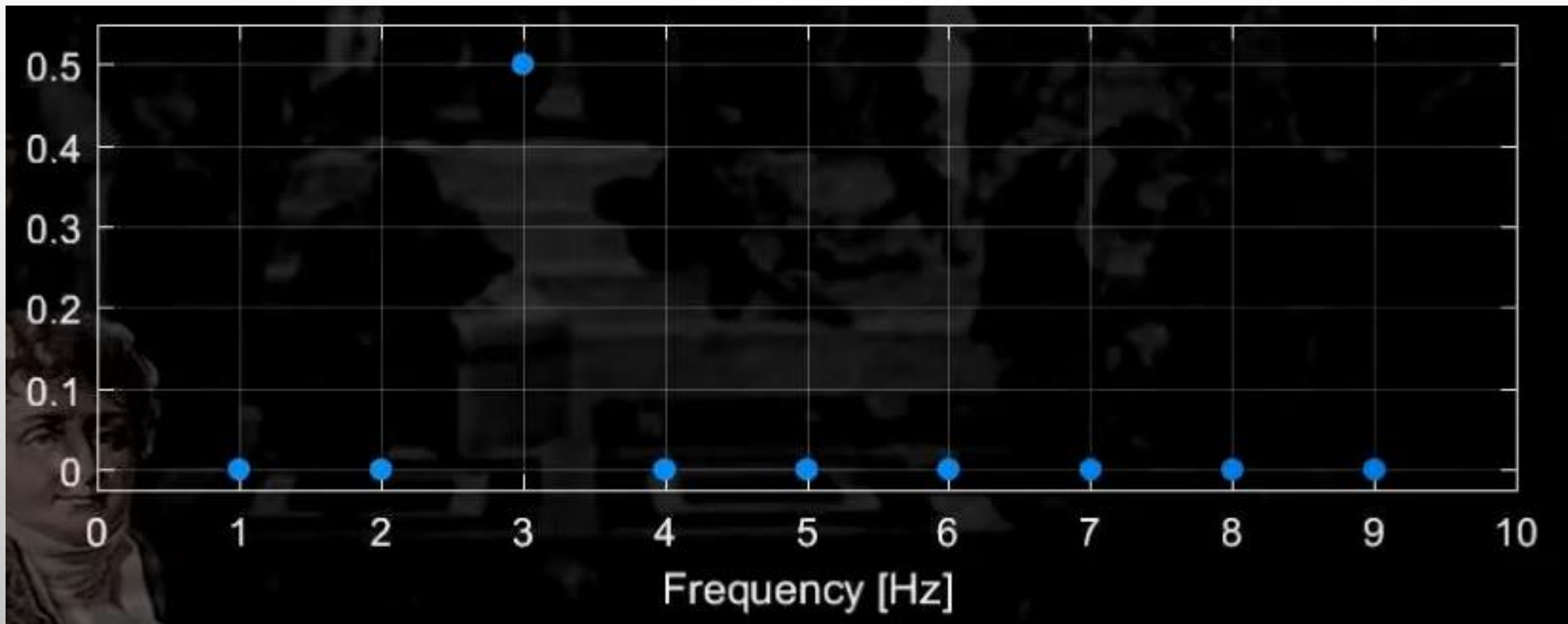
$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) \cos(\omega t) dt + i \int_{-\infty}^{\infty} f(t) \sin(\omega t) dt$$

◆ 원리



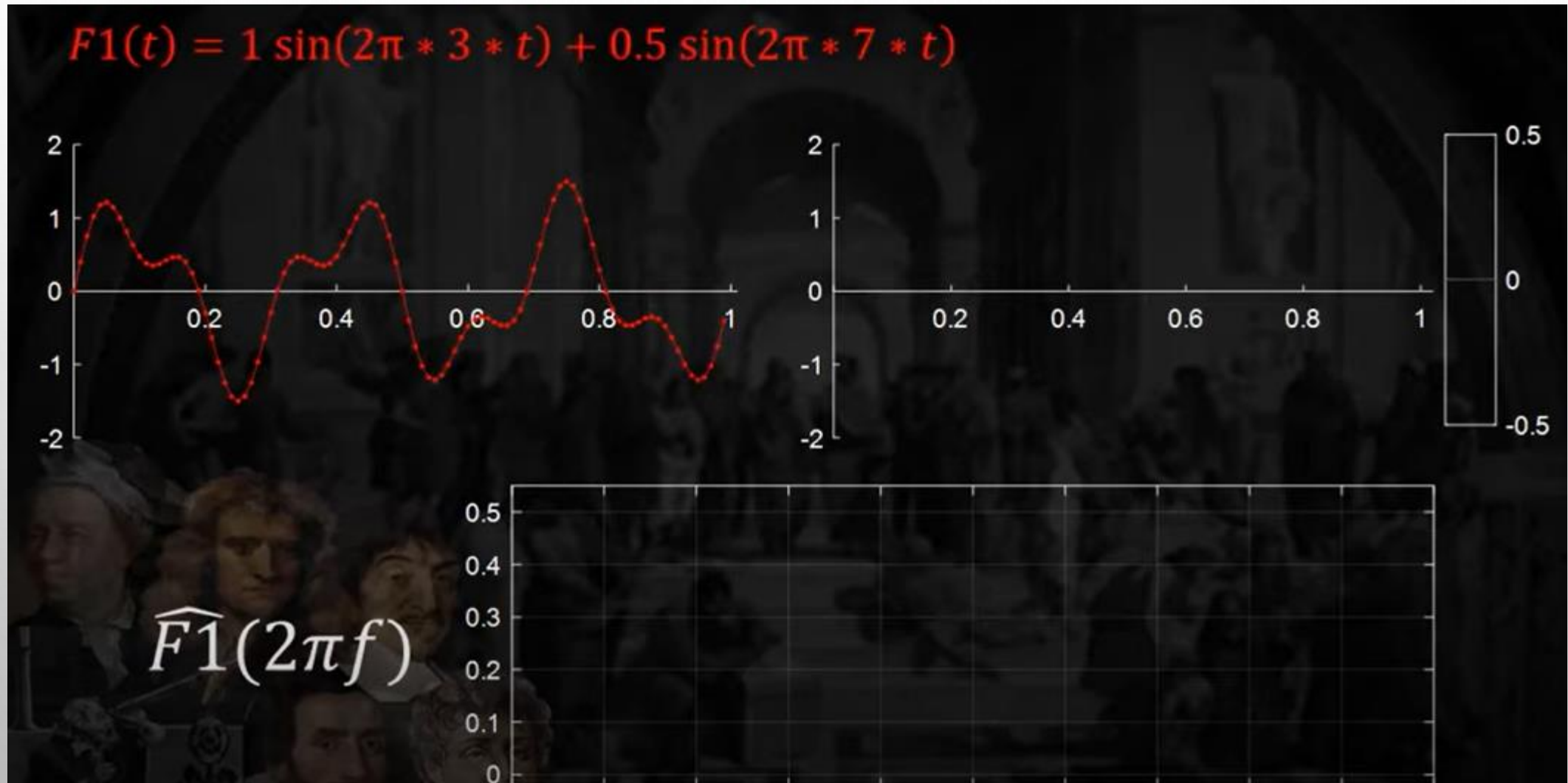
9.2 이산 푸리에 변환

◆ 원리

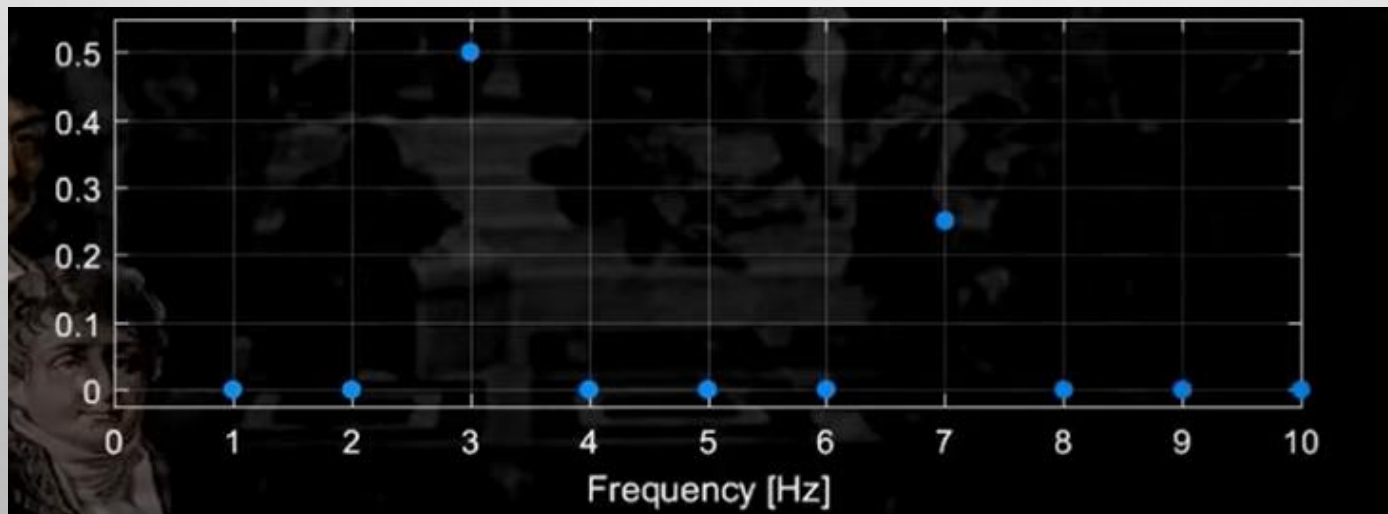
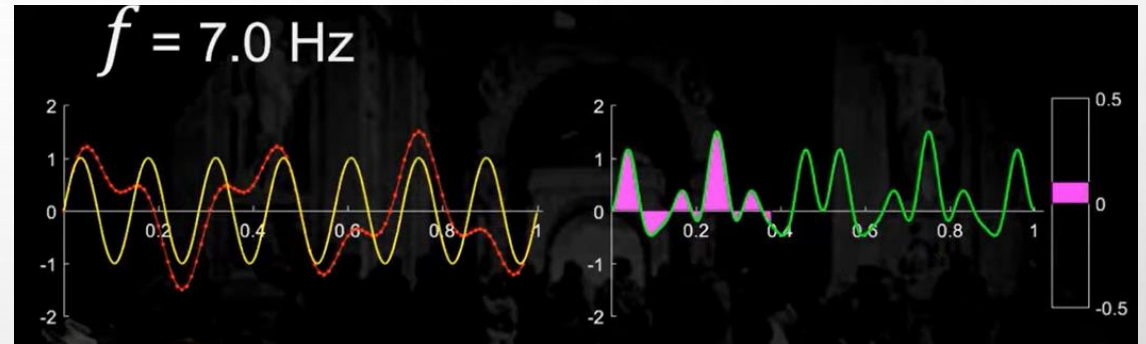
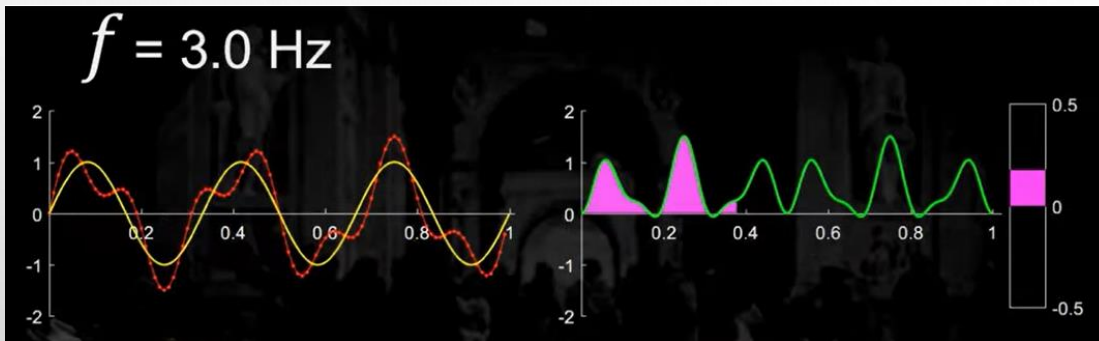
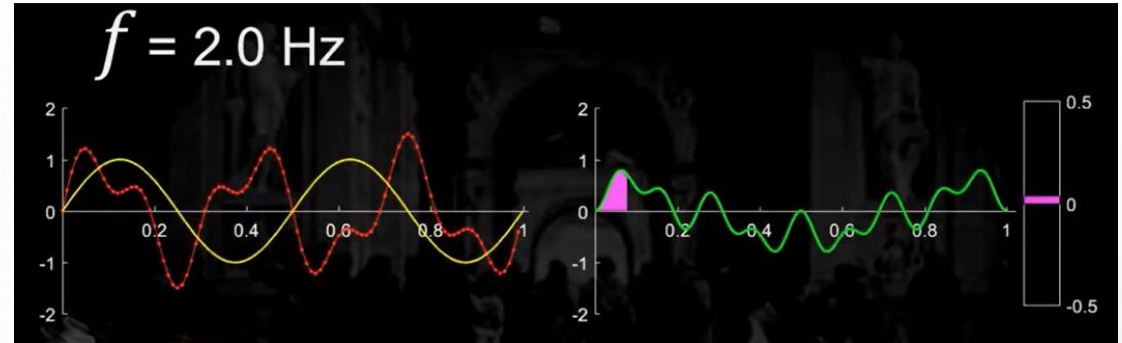
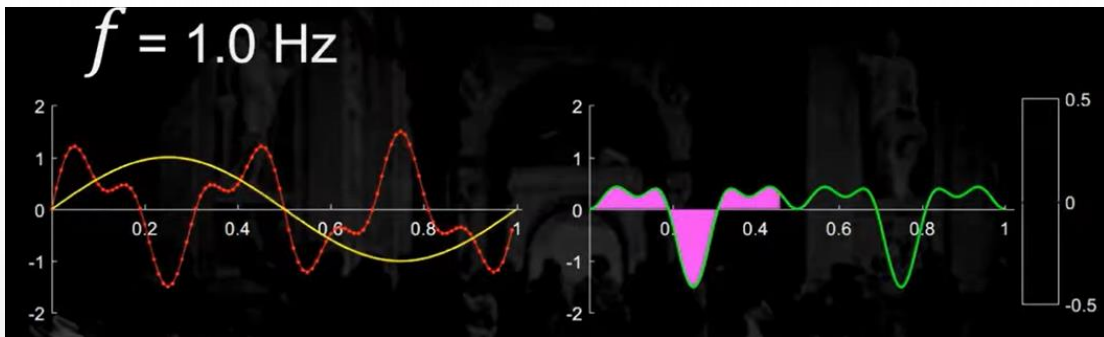


9.2 이산 푸리에 변환

◆ 원리

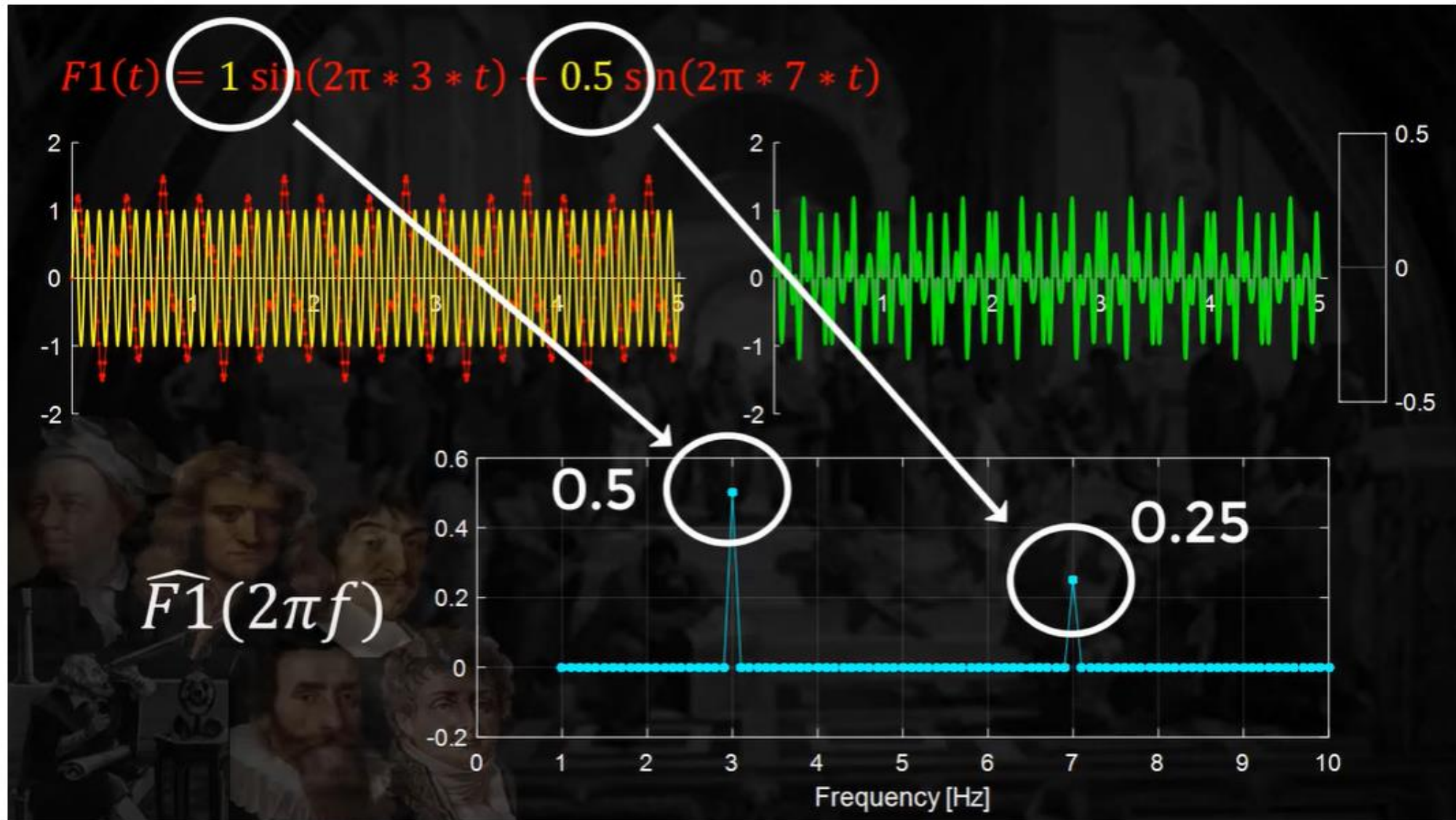


9.2 이산 푸리에 변환



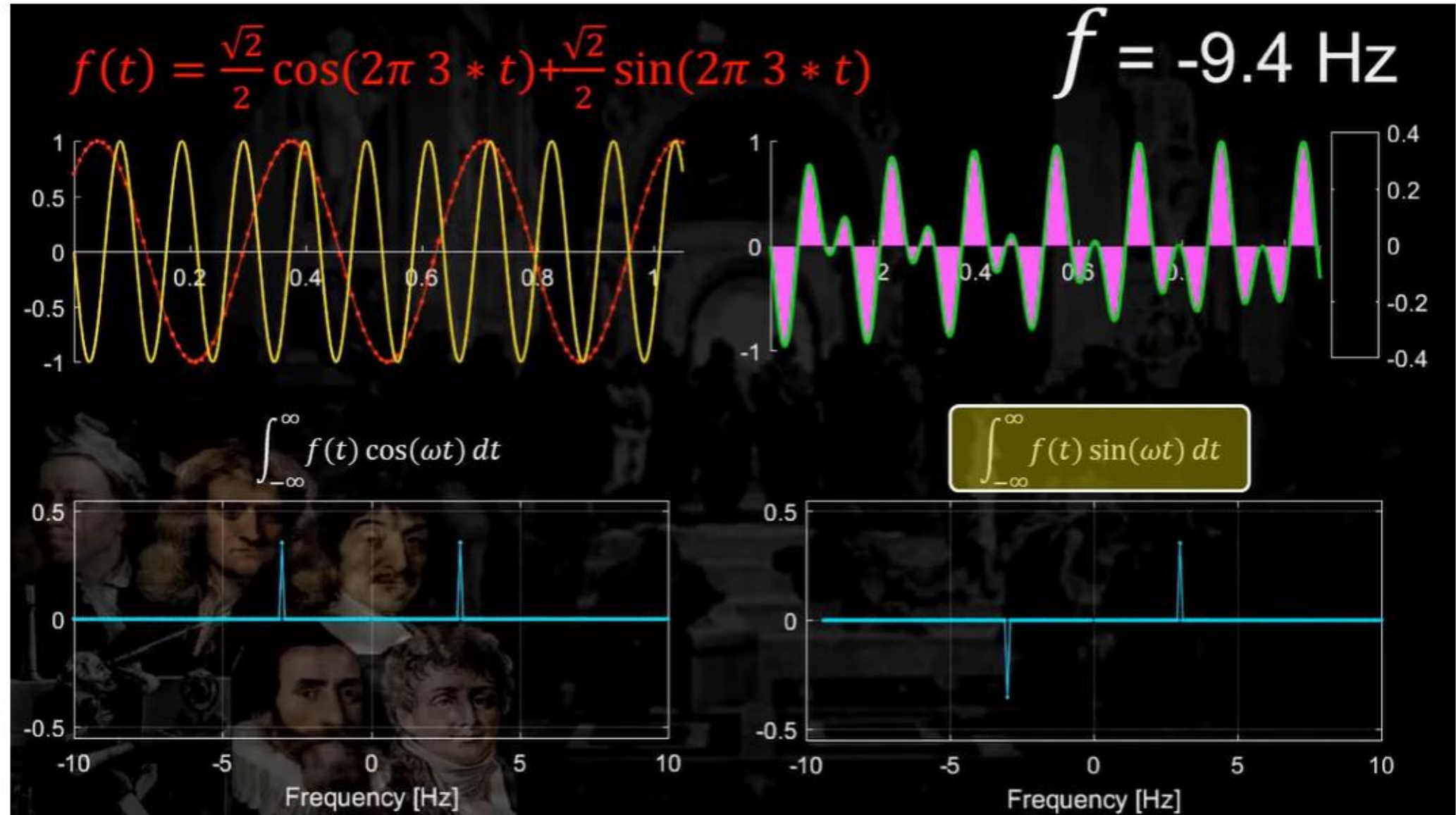
9.2 이산 푸리에 변환

◆ 원리



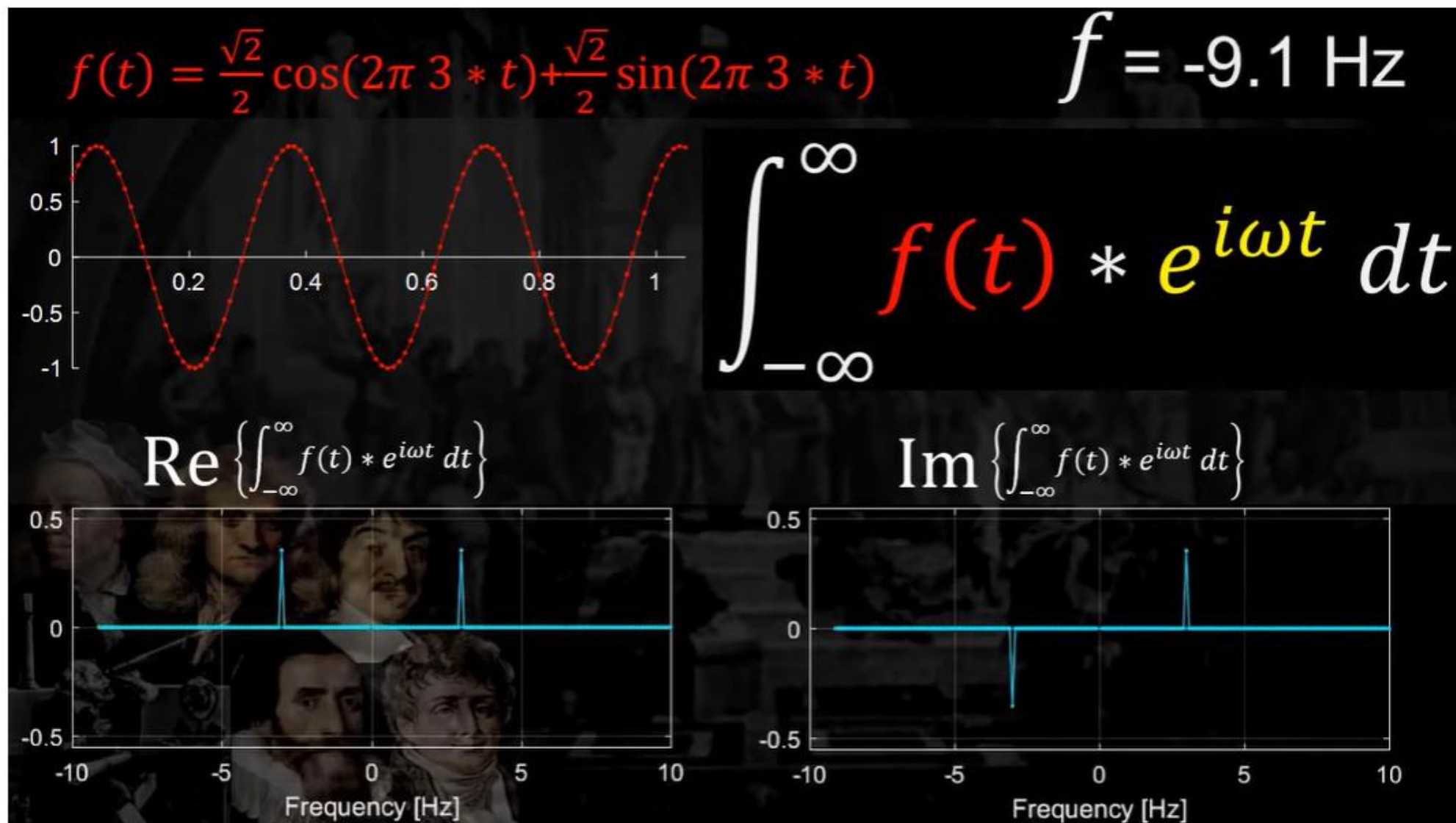
9.2 이산 푸리에 변환

◆ 원리



9.2 이산 푸리에 변환

◆ 원리



9.2 이산 푸리에 변환

◆ 이산 푸리에 변환(DFT: Discrete Fourier Transform)

❖ 디지털 신호(이산신호)에 적용

$$G(k) = \sum_{n=0}^{N-1} g[n] \cdot e^{-j2\pi k \frac{n}{N}}, \quad k=0, \dots, N-1$$
$$g[n] = \frac{1}{N} \sum_{k=0}^{N-1} G(k) \cdot e^{j2\pi k \frac{n}{N}}, \quad n=0, \dots, N-1$$

- $g[n]$: 디지털 신호, $G[k]$: 주파수 k 에 대한 푸리에 변환 계수
- k 와 n 은 신호의 원소개수 (유한개)

9.2 이산 푸리에 변환

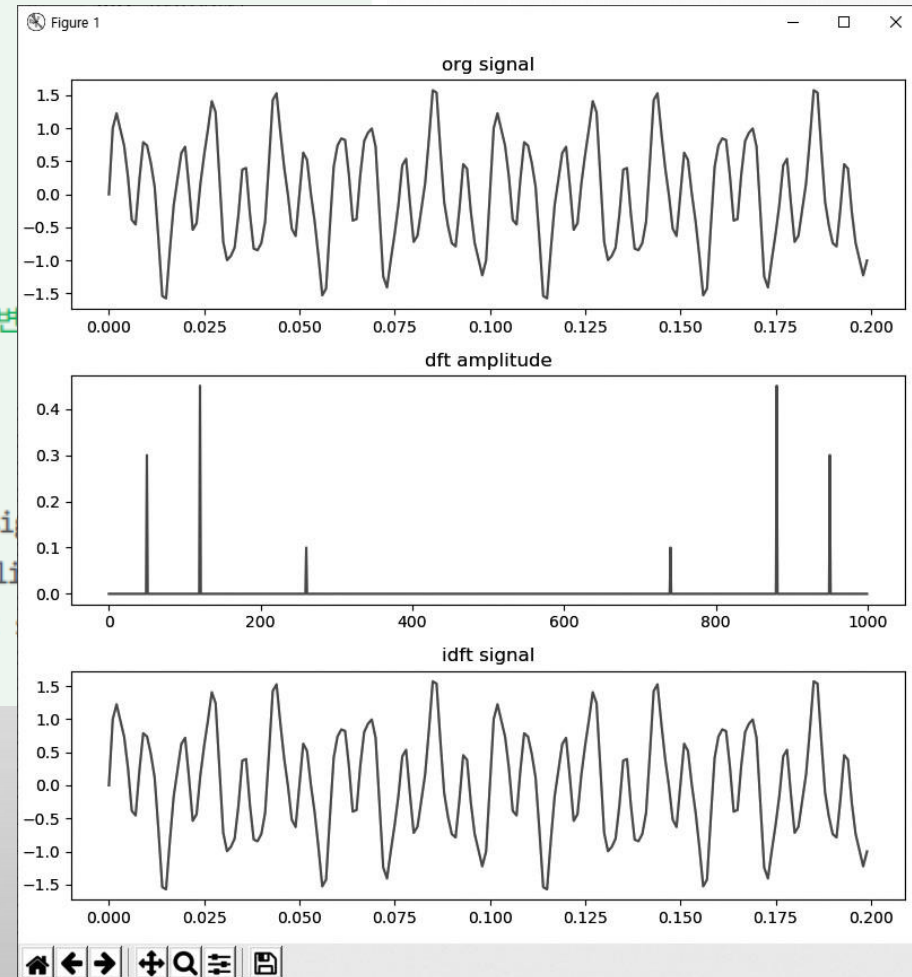
예제 9.2.2

1차원 이산 푸리에 변환 - 03.1d_dft.cpp

```
01 import numpy as np, math
02 import matplotlib.pyplot as plt
03
04 def exp(knN):
05     th = -2 * math.pi * knN                # 푸리에 변환 각도값
06     return complex(math.cos(th), math.sin(th)) # 복소수 클래스
07
08 def dft(g):
09     N = len(g)
10     dst = [sum(g[n] * exp(k*n/N) for n in range(N)) for k in range(N) ]
11     return np.array(dst)
12
13 def idft(g):
14     N = len(g)
15     dst = [sum(g[n] * exp(-k*n/N) for n in range(N)) for k in range(N) ]
16     return np.array(dst) / N
17
```


9.2 이산 푸리에 변환

```
18 fmax = 1000                                # 샘플링 주파수 1000Hz: 최대주파수의 2배
19 dt = 1/fmax                                # 샘플링 간격
20 t = np.arange(0, 1, dt)                    # Time vector
21
22 g1 = np.sin(2 * np.pi * 50 * t)
23 g2 = np.sin(2 * np.pi * 120 * t)
24 g3 = np.sin(2 * np.pi * 260 * t)
25 g = g1 * 0.6 + g2 * 0.9 + g3 * 0.2        # 신호 합성
26
27 N = len(g)                                  # 신호 길이
28 df = fmax/N                                # 샘플링 간격
29 f = np.arange(0, N, df)
30 xf = dft(g) * dt                            # 저자구현 푸리에 변
31 g2 = idft(xf) / dt
32
33 plt.figure(figsize=(10,10))
34 plt.subplot(3,1,1), plt.plot(t[0:200], g[0:200]), plt.title("org si
35 plt.subplot(3,1,2), plt.plot(f, np.abs(xf) ), plt.title("dft ampli
36 plt.subplot(3,1,3), plt.plot(t[0:200], g2[0:200]), plt.title("idft
37 plt.show()
```



9.2 이산 푸리에 변환

◆ 2차원 이산 푸리에 변환

$$\begin{aligned} G(k, l) &= \sum_{m=0}^{M-1} \left(\sum_{n=0}^{N-1} g[n, m] \cdot e^{-j2\pi k \frac{n}{N}} \right) \cdot e^{-j2\pi l \frac{m}{M}}, & k=0, \dots, N-1 \\ & & l=0, \dots, M-1 \\ &= \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} g[n, m] \cdot e^{-j2\pi \left(\frac{kn}{N} + \frac{lm}{M} \right)} \end{aligned}$$

◆ 2차원 이산 푸리에 역변환

$$\begin{aligned} g[n, m] &= \frac{1}{NM} \cdot \sum_{m=0}^{M-1} \left(\sum_{n=0}^{N-1} G(k, l) \cdot e^{j2\pi k \frac{n}{N}} \right) \cdot e^{j2\pi l \frac{m}{M}}, & k=0, \dots, N-1 \\ & & l=0, \dots, M-1 \\ &= \frac{1}{NM} \cdot \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} G(k, l) \cdot e^{j2\pi \left(\frac{kn}{N} + \frac{lm}{M} \right)} \end{aligned}$$

9.2 이산 푸리에 변환

◆ 사인과 코사인 함수로 변경

$$G(k) = \sum_{n=0}^{N-1} g[n] \cdot \left(\cos\left(-2\pi k \frac{n}{N}\right) + j \cdot \sin\left(-2\pi k \frac{n}{N}\right) \right)$$

◆ 실수부와 허수부를 구분하여 표현 → 복소수 행렬 생성됨

$$G(k)_{Re} = \sum_{n=0}^{N-1} g[n]_{Re} \cdot \cos\left(-2\pi k \frac{n}{N}\right) - g[n]_{Im} \cdot \sin\left(-2\pi k \frac{n}{N}\right)$$

$$G(k)_{Im} = \sum_{n=0}^{N-1} g[n]_{Im} \cdot \cos\left(-2\pi k \frac{n}{N}\right) + g[n]_{Re} \cdot \sin\left(-2\pi k \frac{n}{N}\right)$$

$$g[n]_{Re} = \frac{1}{N} \sum_{k=0}^{N-1} G(k)_{Re} \cdot \cos\left(2\pi k \frac{n}{N}\right) - G(k)_{Im} \cdot \sin\left(2\pi k \frac{n}{N}\right)$$

$$g[n]_{Im} = \frac{1}{N} \sum_{k=0}^{N-1} G(k)_{Im} \cdot \cos\left(2\pi k \frac{n}{N}\right) + G(k)_{Re} \cdot \sin\left(2\pi k \frac{n}{N}\right)$$

푸리에 변환

푸리에 역변환

9.2 이산 푸리에 변환

◆ 복소수의 행렬을 확인하려면 → 영상으로 표현

❖ 복소수의 실수부와 허수부를 벡터로 간주하여 벡터의 크기 계산 → 주파수 스펙트럼

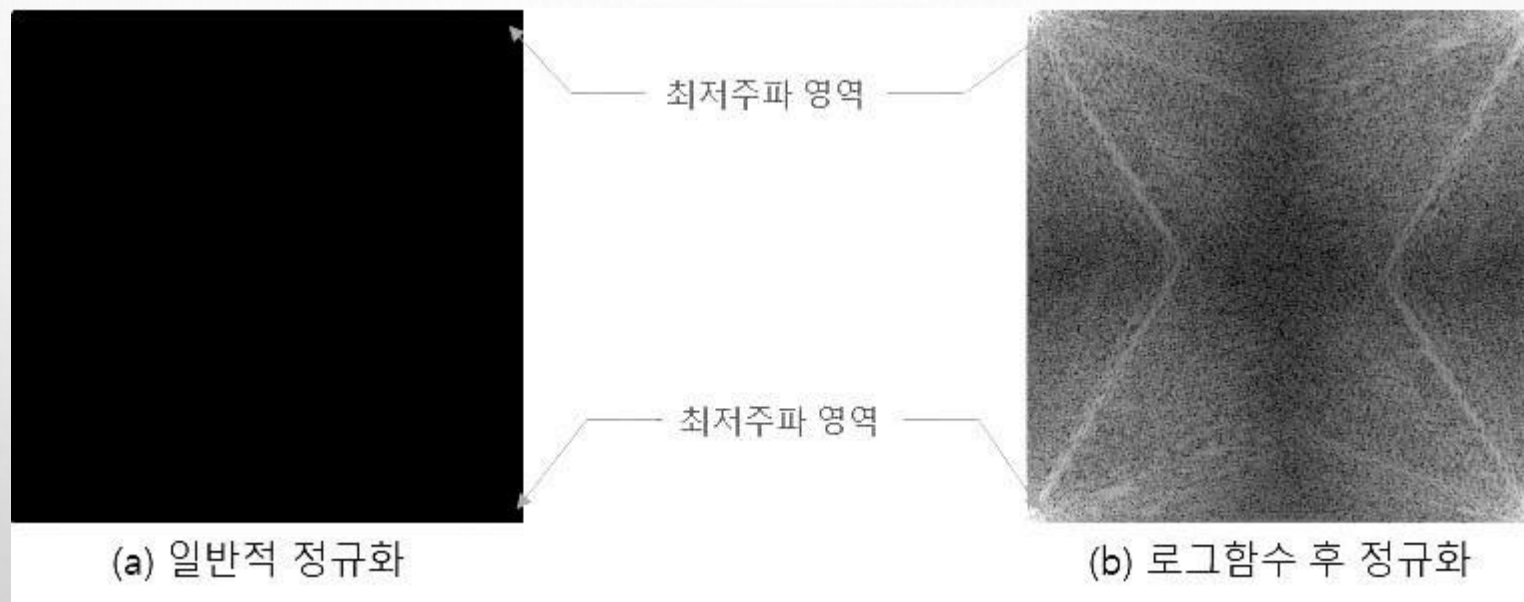
❖ 실수부와 허수부 원소의 기울기 계산 → 주파수 위상

$$|G(k, l)| = \sqrt{Re(k, l)^2 + Im(k, l)^2}$$
$$\theta(k, l) = \tan^{-1} \left[\frac{Im(k, l)}{Re(k, l)} \right]$$

9.2 이산 푸리에 변환

◆ 주파수 스펙트럼 영상

- ❖ 저주파 영역의 계수값이 고주파 영역에 비해 상대적으로 너무 큼
- ❖ 계수값을 정규화해서 영상으로 표현하면 최저주파 영역만 흰색으로 나타나고 나머지 영역은 거의 검은색으로 나타남 → 계수값에 로그함수 적용



9.2 이산 푸리에 변환

◆로그 적용 정규화 함수

```
01 def calc_spectrum(complex):  
02     if complex.ndim==2:  
03         dst = abs(complex) # sqrt(re^2 + im^2) 계산해줌  
    else:  
        dst = cv2.magnitude(complex[:, :, 0], complex[:, :, 1]) # OpenCV의 경우  
06     dst = cv2.log(dst + 1)  
07     cv2.normalize(dst, dst, 0, 255, cv2.NORM_MINMAX)  
08     return cv2.cvtColor(dst, cv2.COLOR_GRAY2BGR)
```

복소수 객체 행렬 사용 - 2차원 행렬

복소수를 2채널로 구성
- 3차원 행렬

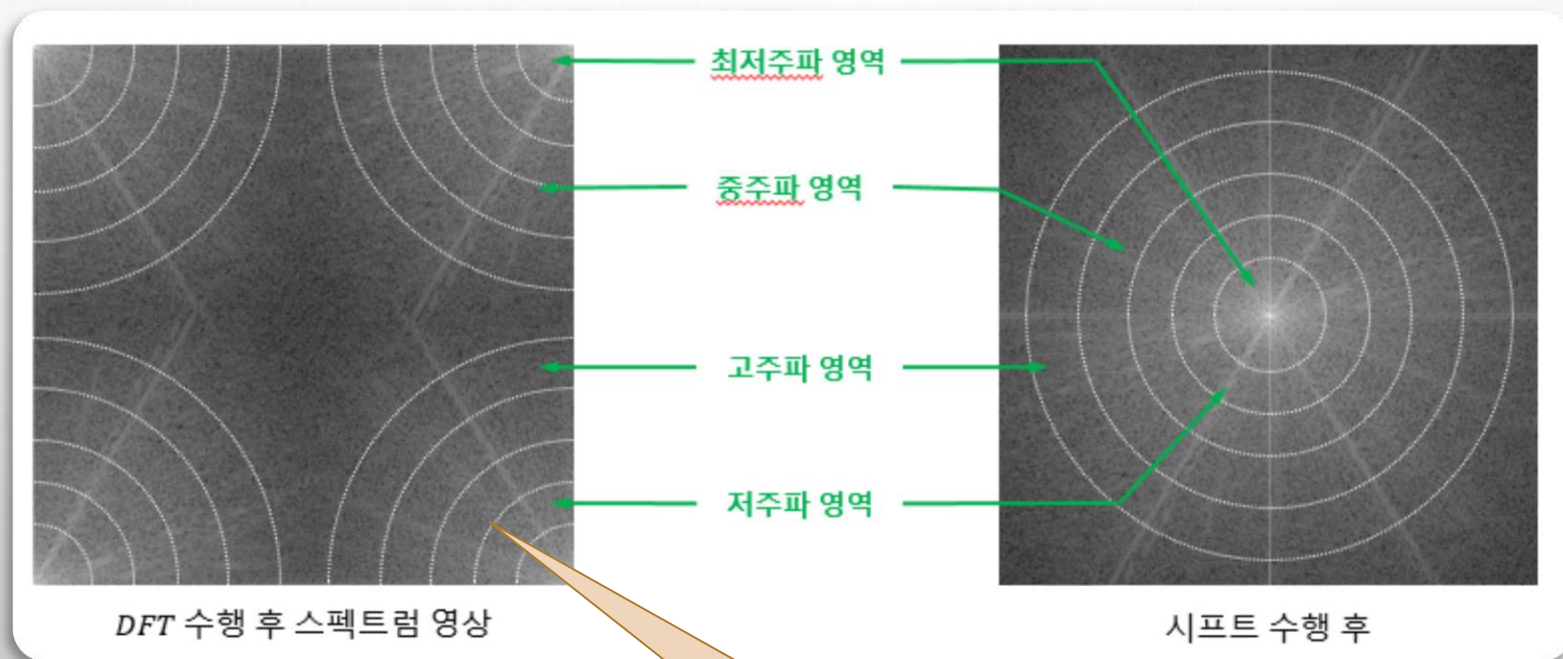
log 함수는 0에서
무한대이기 때문에 1을 더함

윈도우 표시 위해 uchar 형변환

9.2 이산 푸리에 변환

◆ 주파수 스펙트럼 영상

- ❖ 저주파 영역이 모서리 부분에 위치, 고주파 부분이 중심부에 위치
- ❖ 사각형의 각 모서리를 중심으로 원형의 밴드를 형성하여 주파수 영역 분포
 - 해당 주파수 영역 처리시 불편함 → 모양 변경 필요



실제 표시 되지않는 선 -
설명을 위해 표시

9.2 이산 푸리에 변환

◆ 해결 방법 - 셔플링(shuffling)

- ❖ 1사분면과 3사분면의 영상 맞교환, 2사분면과 4사분면의 영상 맞교환
 - 푸리에 변환 주파수 스펙트럼의 주기가 180도이기 때문에 가능
- ❖ 중심이 최저주파 영역, 바깥쪽으로 갈수록 고주파 영역
 - 원형의 밴드로 주파수 영역 쉽게 구분 가능

9.2 이산 푸리에 변환

◆ 셔플링 함수 구현

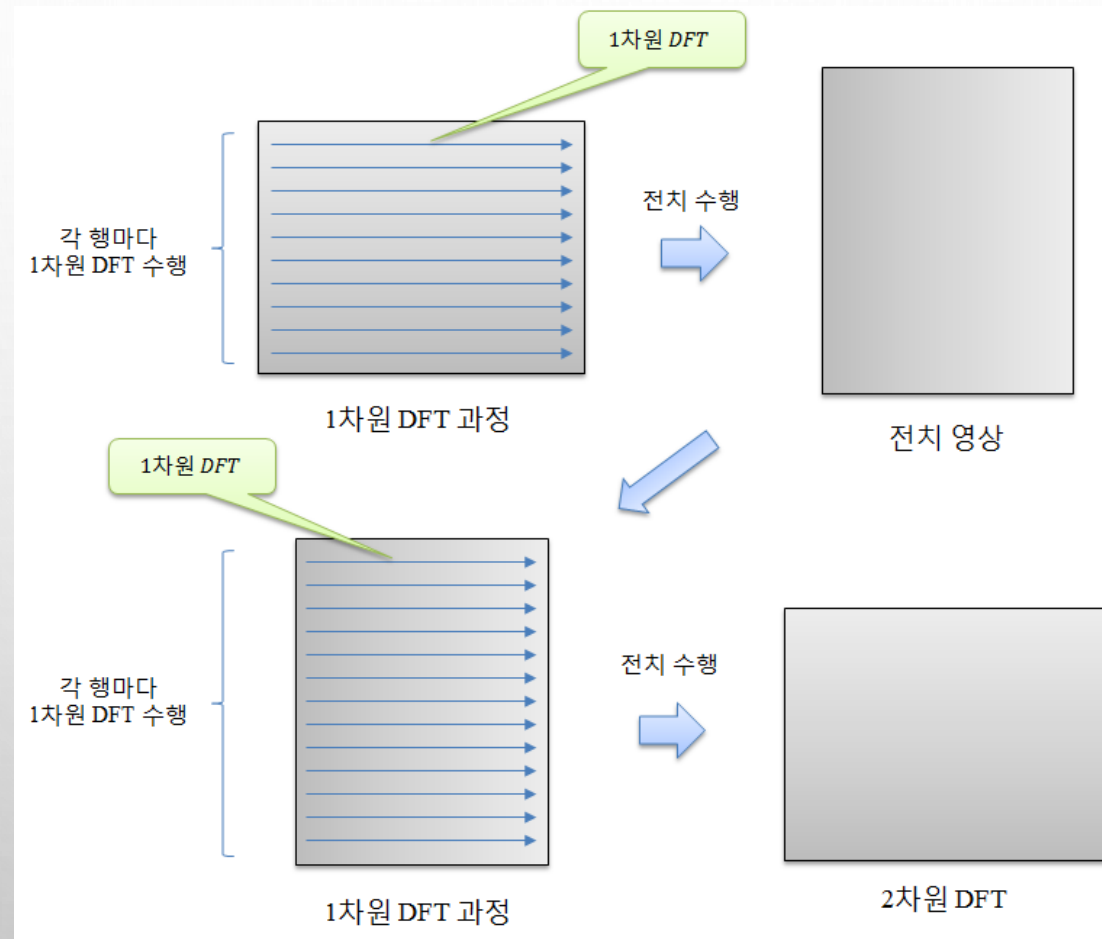
```
def fftshift(img):  
    dst = np.zeros(img.shape, img.dtype)  
    h, w = dst.shape[:2]  
    cy, cx = h // 2, w // 2  
    dst[h-cy:, w-cx:] = np.copy(img[0:cy, 0:cx ]) # 뒀연산자  
    dst[0:cy, 0:cx ] = np.copy(img[h-cy:, w-cx:]) # 1사분면→ 3사분면  
    dst[0:cy, w-cx:] = np.copy(img[h-cy:, 0:cx ]) # 3사분면→ 1사분면  
    dst[h-cy:, 0:cx ] = np.copy(img[0:cy, w-cx:]) # 2사분면→ 4사분면  
    return dst
```

복사하지 않고 원본 영상을 목적 영상에 할당
하면 참조가 되어 결과에 오류 발생함

9.2 이산 푸리에 변환

◆ 2차원 DFT 수행 방법

❖ 1차원 푸리에 변환 결과 전치 → 다시 1차원 푸리에 변환 수행 → 다시 전치



9.2 이산 푸리에 변환

예제 9.2.3

2차원 이산 푸리에 변환 - 04.2d_dft.py

```
01 import numpy as np, cv2
02 from Common.dft2d import dft, idft, calc_spectrum, fftshift # dft 관련 함수 импорт
03
04 def dft2(image):
05     tmp = [dft(row) for row in image]
06     dst = [dft(row) for row in np.transpose(tmp)]
07     return np.transpose(dst) # 전치 환원 후 반환
08
09 def idft2(image):
10     tmp = [idft(row) for row in image]
11     dst = [idft(row) for row in np.transpose(tmp)]
12     return np.transpose(dst) # 전치 환원 후 반환
13
14 def ck_time(mode=0): # 수행시간 체크 함수
15     global stime # 함수 내부에서 값 유지위해
16     if (mode == 0):
17         stime = time.perf_counter()
18     elif (mode==1):
19         etime = time.perf_counter()
20         print("수행시간 = %.5f sec" % (etime - stime)) # 초 단위 경과 시간
21
22 image = cv2.imread("images/dft_240.jpg", cv2.IMREAD_GRAYSCALE)
23 if image is None: raise Exception("영상파일 읽기 에러")
```

1차원 DFT 2번 수행

전치 환원 후 반환

전치 환원 후 반환

수행시간 체크 함수

함수 내부에서 값 유지위해

시작 시간 측정

종료 시간 측정 및 수행시간 계산

9.2 이산 푸리에 변환

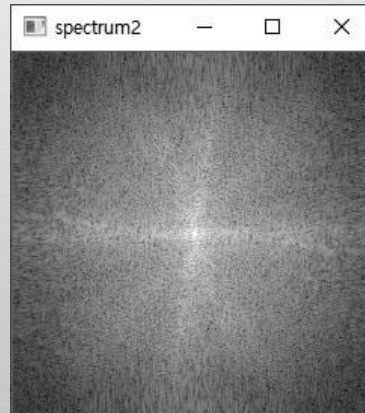
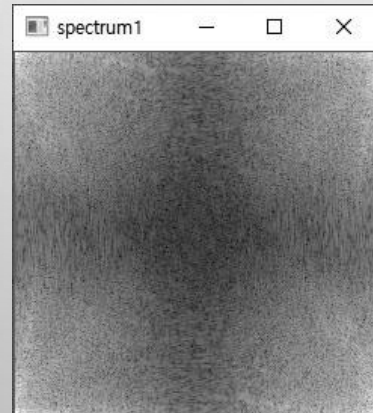
```
25 ck_time(0) # 시작 시간 체크
26 dft = dft2(image) # 2차원 DFT 수행
27 spectrum1 = calc_spectrum(dft) # 주파수 스펙트럼 영상
28 spectrum2 = fftshift(spectrum1) # np.fft.fftshift() 사용 가능
29 idft = idft2(dft).real # 2차원 IDFT 수행
30 ck_time(1) # 종료 시간 체크
31
32 cv2.imshow("image", image)
33 cv2.imshow("spectrum1", spectrum1)
34 cv2.imshow("spectrum2", spectrum2)
35 cv2.imshow("idft_img", cv2.convertScaleAbs(idft))
```

복소 객체의 실수부만 가져옴

◆ 실행결과

Run: 04.2d_dft

C:\Python\python.exe D:/source/chap09/04.2d_dft.py
수행시간 = 157.06352 sec



9.3 고속 푸리에 변환

◆이산 푸리에 변환

- ❖ 원본 신호의 한 원소에 곱해지는 기저 함수의 원소들을 원소 길이만큼 반복적으로 곱해짐
 - 때문에 신호가 커질수록 계산 속도는 기하급수적으로 증가

◆고속 푸리에 변환

- ❖ 삼각함수의 주기성 이용해 작은 단위 분리
- ❖ 반복적 수행하고 합해져서 효율성 높이는 방법

9.3 고속 푸리에 변환

◆ 푸리에 변환 수식 - 짝수부/ 홀수부 분리 가능

$$G(k) = \sum_{n=0}^{L-1} g[2n] \cdot e^{-j2\pi k \frac{2n}{2L}} + \sum_{n=0}^{L-1} g[2n+1] \cdot e^{-j2\pi k \frac{2n+1}{2L}}$$
$$G(k) = \left\{ \sum_{n=0}^{L-1} g[2n] \cdot e^{-j2\pi k \frac{n}{L}} \right\} + \left\{ \sum_{n=0}^{L-1} g[2n+1] \cdot e^{-j2\pi k \frac{n}{L}} \right\} \cdot e^{-j2\pi k \frac{1}{2L}}$$

❖ 짝수부 변환 수식

$$G_{\text{even}}(k) = \sum_{n=0}^{L-1} g[2n] \cdot e^{-j2\pi k \frac{n}{L}}$$

❖ 홀수부 변환 수식

$$G_{\text{odd}}(k) = \sum_{n=0}^{L-1} g[2n+1] \cdot e^{-j2\pi k \frac{n}{L}}$$

◆ 수식의 정리

$$G(k) = G_{\text{even}}(k) + G_{\text{odd}}(k) \cdot e^{-j2\pi k \frac{1}{2L}}$$

9.3 고속 푸리에 변환

◆ 공통 지수부분에서 한 주기(L)를 더한 수식

$$e^{-j2\pi(k+L)\frac{n}{L}} = e^{-j2\pi k\frac{n}{L}} \cdot e^{-j2\pi\frac{Ln}{L}} = e^{-j2\pi k\frac{n}{L}} \cdot e^{-j2\pi n} = e^{-j2\pi k\frac{n}{L}}$$

$$\begin{aligned} G(k+L) &= G_{\text{even}}(k+L) + G_{\text{odd}}(k+L) \cdot e^{-j2\pi(k+L)\frac{1}{2L}} \\ &= G_{\text{even}}(k) + G_{\text{odd}}(k) \cdot e^{-j2\pi(k+L)\frac{1}{2L}} \end{aligned}$$

$$e^{-j2\pi(k+L)\frac{1}{2L}} = e^{-j2\pi k\frac{1}{2L}} \cdot e^{-j2\pi\frac{L}{2L}} = e^{-j2\pi k\frac{1}{2L}} \cdot e^{-j\pi} = -e^{-j2\pi k\frac{1}{2L}}$$

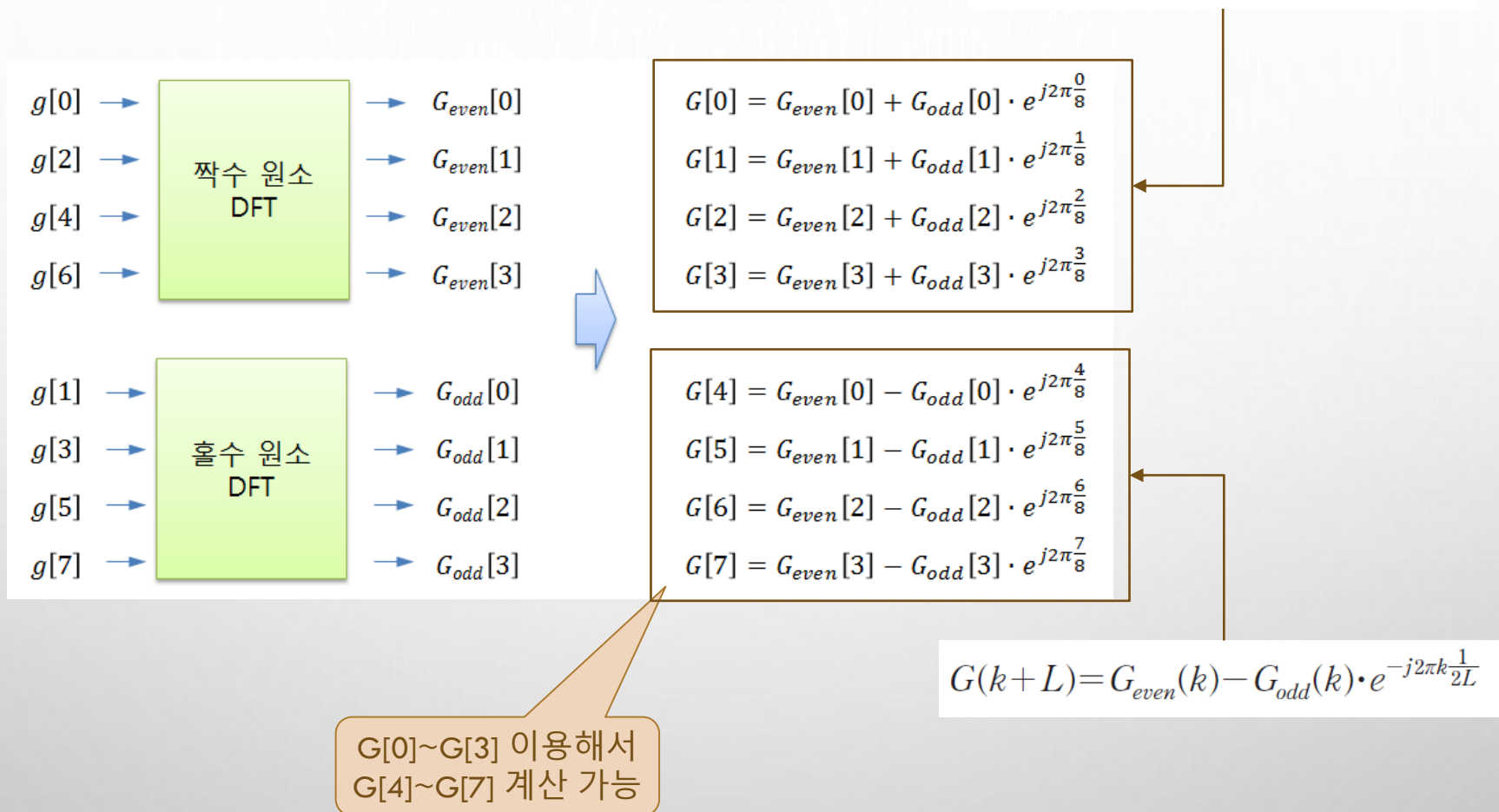
❖ 한 쌍의 $G(k)$ 으로 $G(k+L)$ 의 값 계산 가능 → 속도 향상

$$G(k+L) = G_{\text{even}}(k) - G_{\text{odd}}(k) \cdot e^{-j2\pi k\frac{1}{2L}}$$

9.3 고속 푸리에 변환

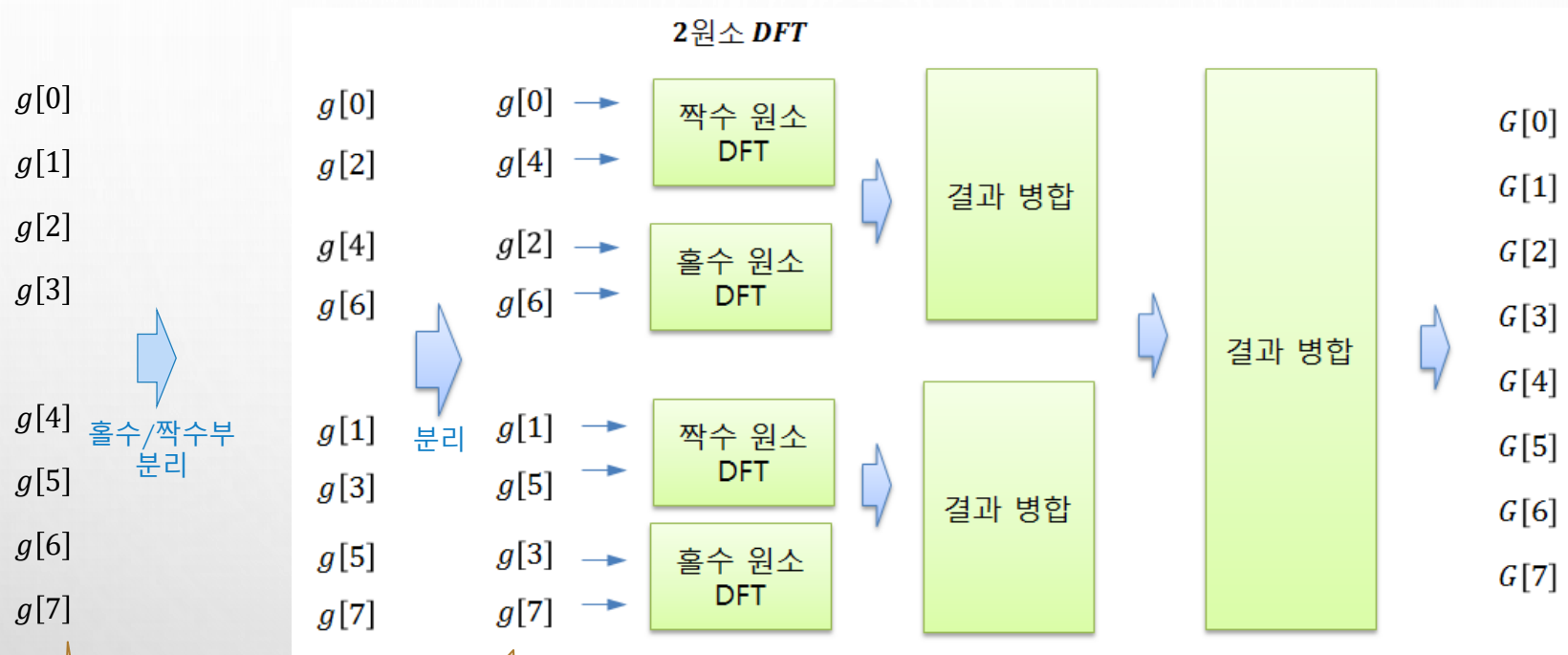
◆ 짝수원소 홀수 원소 분리 및 계산

$$G(k) = G_{\text{even}}(k) + G_{\text{odd}}(k) \cdot e^{-j2\pi k \frac{1}{2L}}$$



9.3 고속 푸리에 변환

◆ 연속적 신호 분리 → 2원소로 분리

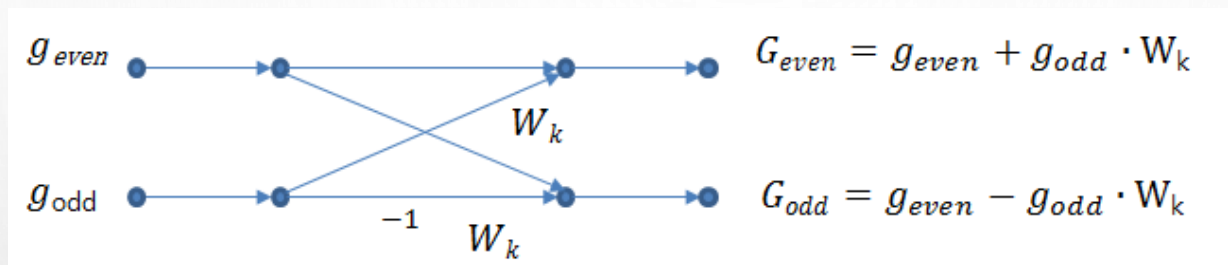


9.3 고속 푸리에 변환

◆버터플라이(butterfly) 과정

❖ 스캔들 결과 원소에서 이웃한 두 원소에 대해서 이산 푸리에 변환 수행

● W_k : 푸리에 변환의 기저함수

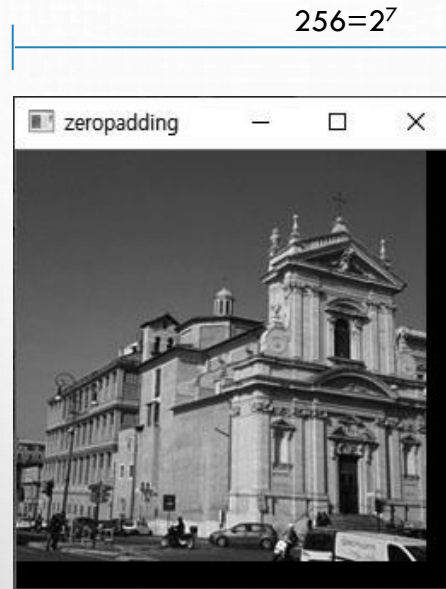


❖ 원본 신호를 연속적으로 짝수부와 홀수부로 분리 → 원본 신호의 원소개수는 2의 자승

9.3 고속 푸리에 변환

◆ 영삽입(zero-padding)

- ❖ 푸리에 변환할 영상의 크기가 반드시 2의 자승 아님
- ❖ 원본 영상의 가로와 세로 크기를 2의 자승이 되도록 넓히는 방법



빈공간 검은색(0)으로 채움

9.3 고속 푸리에 변환

❖ 영삽입 함수 구현

```
def zeropadding(img):  
    h, w = img.shape[:2]  
    m = 1 << int(np.ceil(np.log2(h)))           # 2의 자승 계산  
    n = 1 << int(np.ceil(np.log2(w)))  
    dst = np.zeros((m, n), img.dtype)          # 2의 자승 크기 영상 생성  
    dst[0:h, 0:w] = img[:]                     # 자승 크기 영상에 원본 영상 복사  
    return dst
```

9.3 고속 푸리에 변환

예제 9.3.1

고속 푸리에 변환 - 05.2d_fft.py

```
01 import numpy as np, cv2
02 from Common.dft2d import exp, calc_spectrum, fftshift      # dft 관련함수 импорт
03 from Common.fft2d import zeropadding                      # 영삽입 함수 импорт
04
05 def butterfly(pair, L, N, dir):                            # 버터플라이 수행 함수
06     for k in range(L):
07         Geven, Godd = pair[k], pair[k + L]
08         pair[k]      = Geven + Godd * exp(dir * k / N)    # 짝수부
09         pair[k + L]  = Geven - Godd * exp(dir * k / N)    # 홀수부
10
11 def parring(g, N, dir, start=0, stride=1):                # 2원소까지 재귀 통한 분리&합성
12     if N == 1: return [g[start]]
13     L = N // 2
14     sd = stride * 2
15     part1 = parring(g, L, dir, start, sd)                 # 홀수 신호 재귀 분리
16     part2 = parring(g, L, dir, start + stride, sd)        # 짝수 신호 재귀 분리
17     pair = part1 + part2                                   # 재귀 결과 병합
18     butterfly(pair, L, N, dir)                            # 버터플라이 수행
19     return pair
```

재호출할 때마다
신호길이는 $\frac{1}{2}$, 폭은 2배

재귀호출을 통한 2원소까지 분리

9.3 고속 푸리에 변환

```
21 def fft(g):                                     # 1차원 fft 수행
22     return pairing(g, len(g), 1)
23
24 def ifft(g):                                     # 1차원 ifft 수행
25     fft = pairing(g, len(g), -1)
26     return [v / len(g) for v in fft]
27
28 def fft2(image):
29     pad_img = zeropadding(image)                 # 영삽입
30     tmp = [fft(row) for row in pad_img]
31     dst = [fft(row) for row in np.transpose(tmp)] # 전치 후 1차원 푸리에 변환
32     return np.transpose(dst)                     # 전치 환원 후 반환
33
34 def ifft2(image):
35     tmp = [ifft(row) for row in image]
36     dst = [ifft(row) for row in np.transpose(tmp)] # 전치 후 1차원 푸리에 변환
37     return np.transpose(dst)
38
39 image = cv2.imread("images/dft_240.jpg", cv2.IMREAD_GRAYSCALE)
40 if image is None: raise Exception("영상파일 읽기 에러")
```

역변환 수식따라 신호길이를 나눈값 반환

9.3 고속 푸리에 변환

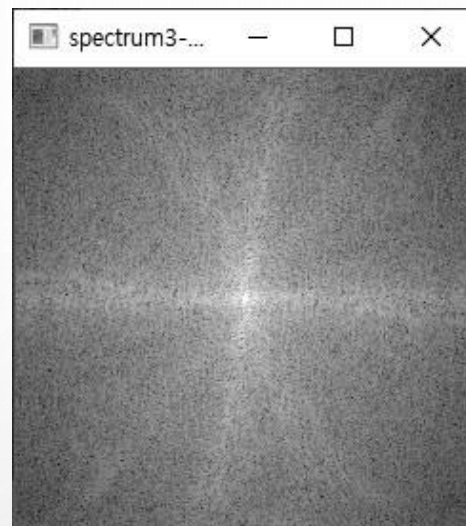
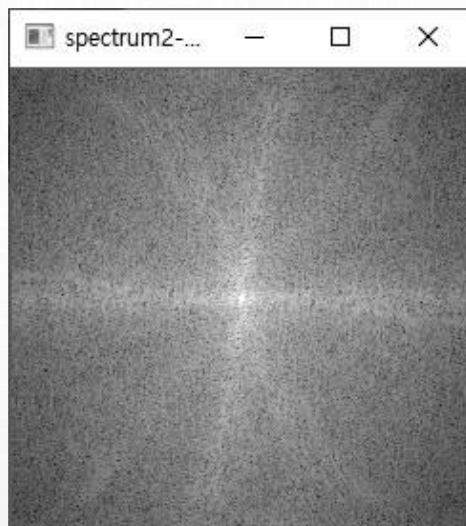
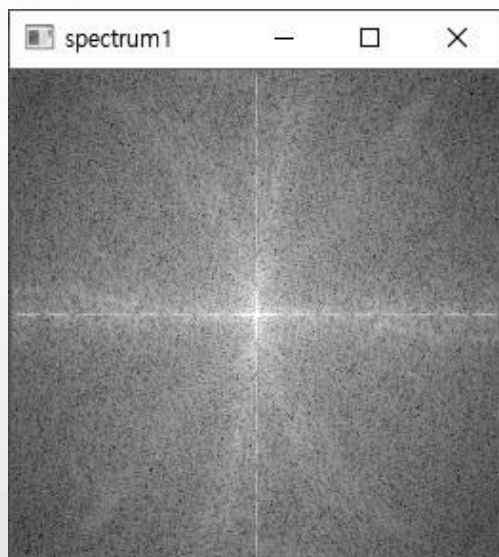
```
42 dft1 = fft2(image) # 저자 구현 fft 함수
43 dft2 = np.fft.fft2(image) # numpy fft 함수
44 dft3 = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT) # OpenCV fft 함수
45
46 spectrum1 = calc_spectrum(fftshift(dft1)) # 시프트후 주파수 스펙트럼 영상 생성
47 spectrum2 = calc_spectrum(fftshift(dft2))
48 spectrum3 = calc_spectrum(fftshift(dft3))
49
50 idft1 = fft2(dft1).real # 저자 구현 fft 역변환
51 idft2 = np.fft.ifft2(dft1).real # numpy fft 역변환
52 idft3 = cv2.idft(dft3, flags=cv2.DFT_SCALE)[:,:,:0] # OpenCV fft 역변환
53
54 print("user 방법 변환 행렬 크기:", dft1.shape) # 푸리에 변환 결과 행렬 형태
55 print("np.fft 방법 변환 행렬 크기:", dft2.shape)
56 print("cv2.dft 방법 변환 행렬 크기:", dft3.shape)
57
58 cv2.imshow("spectrum1", spectrum1) # 주파수 스펙트럼 영상
59 cv2.imshow("spectrum2-np.fft", spectrum2)
60 cv2.imshow("spectrum3-OpenCV", spectrum3)
61 cv2.imshow("idft_img1", cv2.convertScaleAbs(idft1)) # 역변환 후 환원 영상
62 cv2.imshow("idft_img2", cv2.convertScaleAbs(idft2))
63 cv2.imshow("idft_img3", cv2.convertScaleAbs(idft3))
64 cv2.waitKey(0)
```

0번 채널이 실수부

uchar 형 스케일 - 영상 표시 위해

9.3 고속 푸리에 변환

◆ 실행결과



영상입



9.4 FFT를 이용한 주파수 영역 필터링

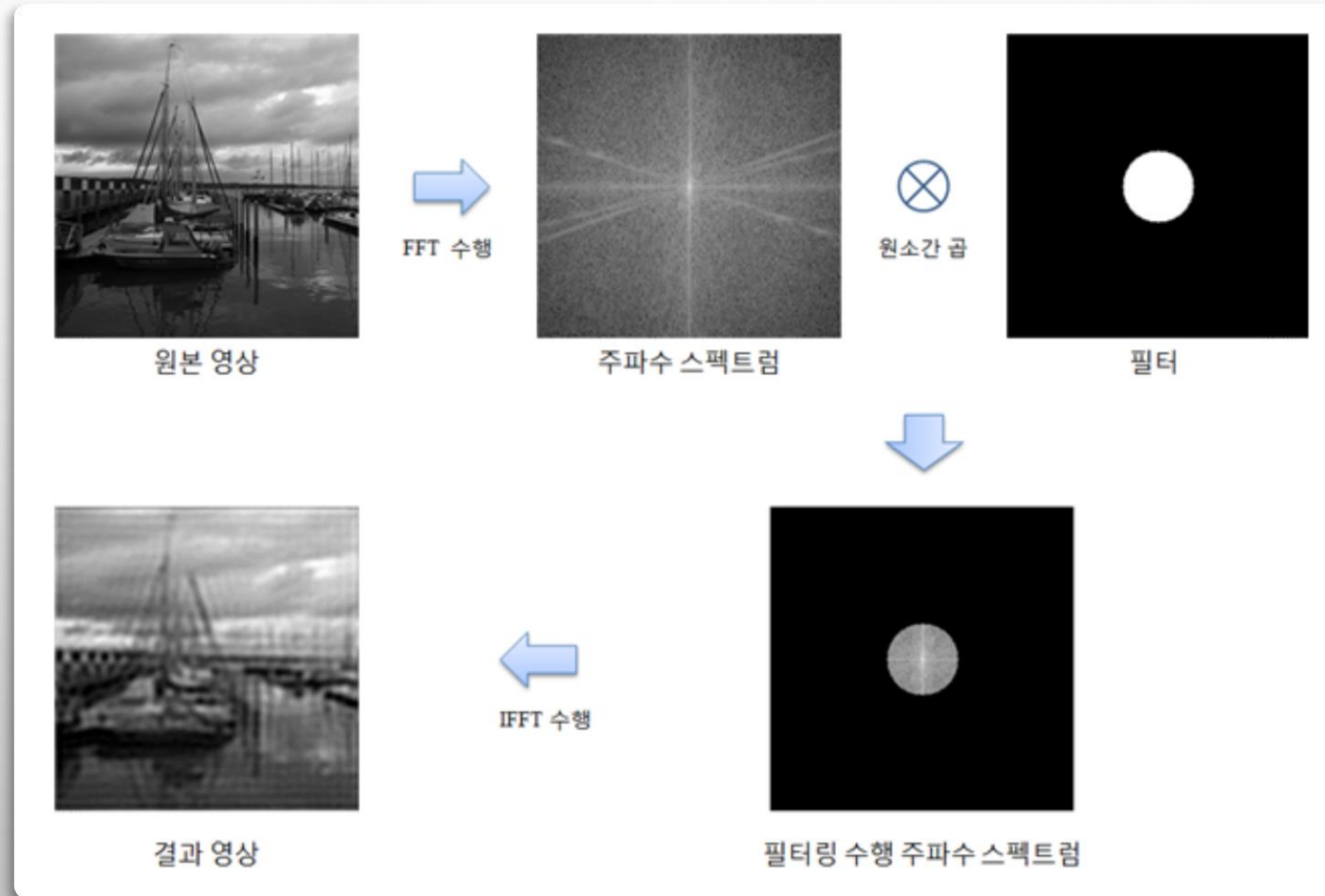
- ◆ 9.4.1 주파수 영역 필터링의 과정
- ◆ 9.4.2 저주파 및 고주파 통과 필터링
- ◆ 9.4.3 버터워스, 가우시안 필터링

9.4.1 주파수 영역 필터링의 과정

◆영상의 주파수 영역 변환

❖ 저주파 영역과 고주파 영역 분리 → 특정 주파수 영역 강화, 약화, 제거 가능

◆주파수 영역 필터링 과정



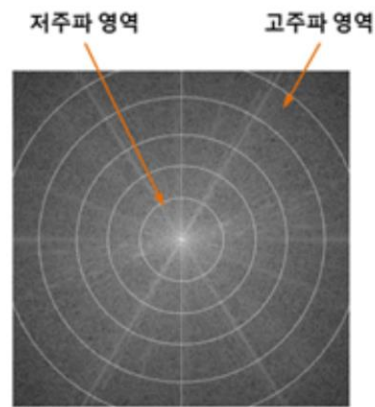
9.4.2 저주파 및 고주파 통과 필터링

❖ 저주파 통과 필터링

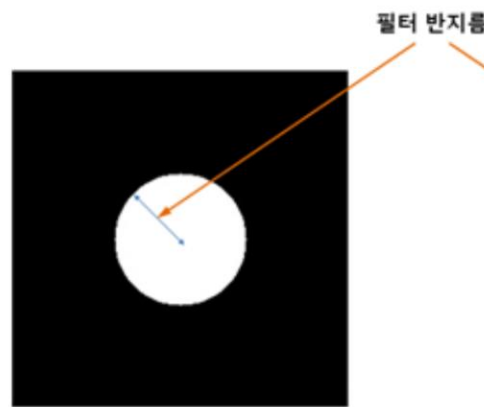
- DFT 변환 영역에서 저주파 영역의 계수들은 통과, 고주파 영역의 계수 차단

❖ 고주파 통과 필터링

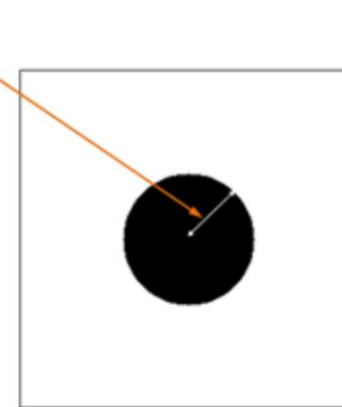
- 고주파 영역의 계수들을 통과시키고 저주파 영역의 계수들 차단



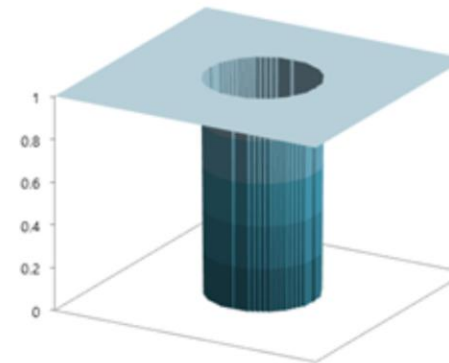
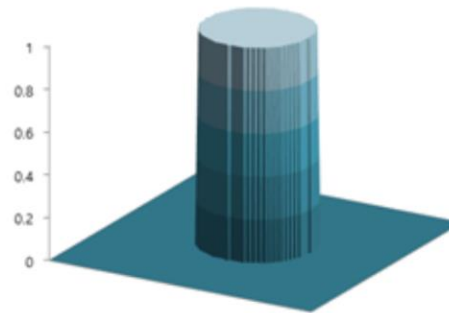
DFT & 서플링



저주파 통과 필터



고주파 통과 필터



9.4.2 저주파 및 고주파 통과 필터링

예제 9.4.1

주파수 영역 필터링1 - 06.FFT_filtering1.py

```
01 import numpy as np, cv2
02 from Common.fft2d import fft2, ifft2, calc_spectrum, fftshift # fft 관련 함수 импорт
03
04 def FFT(image, mode=2):
05     if mode == 1: dft = fft2(image) # 저자 구현 함수
06     elif mode==2: dft = np.fft.fft2(image) # 넘파이 함수
07     elif mode==3: dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
08     dft = fftshift(dft) # 주파수 시프트
09     spectrum = calc_spectrum(dft) # 주파수 스펙트럼 영상
10     return dft, spectrum
11
12 def IFFT(dft, shape, mode=2):
13     dft = fftshift(dft) # 역 시프트
14     if mode == 1: img = ifft2(dft).real
15     if mode == 2: img = np.fft.ifft2(dft).real
16     if mode == 3: img = cv2.idft(dft, flags=cv2.DFT_SCALE)[:,:,0]
17     img = img[:shape[0], :shape[1]] # 영상입 부분 제거
18     return cv2.convertScaleAbs(img) # 절대값 및 uint8 스케일링
19
```

fft 수행 + 시프트 + 스펙트럼 영상 생성

역시프트+ 역fft 수행

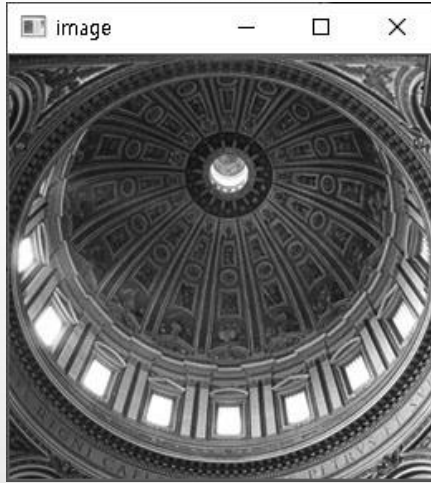
9.4.2 저주파 및 고주파 통과 필터링

```
20 image = cv2.imread("images/filter.jpg", cv2.IMREAD_GRAYSCALE)
21 if image is None: raise Exception("영상파일 읽기 에러")
22 cy, cx = np.divmod(image.shape, 2)[0] # 행렬 중심점 구하기
23 mode = 3 # FFT 방법 선택
24
25 dft, spectrum = FFT(image, mode) # FFT 수행 및 서플링
26 lowpass = np.zeros(dft.shape, np.float32) # 저주파 통과 필터
27 highpass = np.ones(dft.shape, np.float32) # 고주파 통과 필터
28 cv2.circle(lowpass, (cx, cy), 30, (1, 1), -1) # 2개 채널로 값 지정
29 cv2.circle(highpass, (cx, cy), 30, (0, 0), -1)
30
31 lowpassed_dft = dft * lowpass # 주파수 필터링
32 highpassed_dft = dft * highpass # 필터링 - 주파수 계수와 필터행렬의 곱
33 lowpassed_img = IFFT(lowpassed_dft, image.shape, mode) # 푸리에 역변환
34 highpassed_img = IFFT(highpassed_dft, image.shape, mode)
35
36 cv2.imshow("image", image)
37 cv2.imshow("lowpassed_img", lowpassed_img) # 역푸리에 변환 영상
38 cv2.imshow("highpassed_img", highpassed_img)
39 cv2.imshow("spectrum_img", spectrum)
40 cv2.imshow("lowpass_spect", calc_spectrum(lowpassed_dft)) # 필터링 후 스펙트럼
41 cv2.imshow("highpass_spect", calc_spectrum(highpassed_dft))
42 cv2.waitKey(0)
```

OpenCV 함수는 필터 행렬 2개 채널로 구성

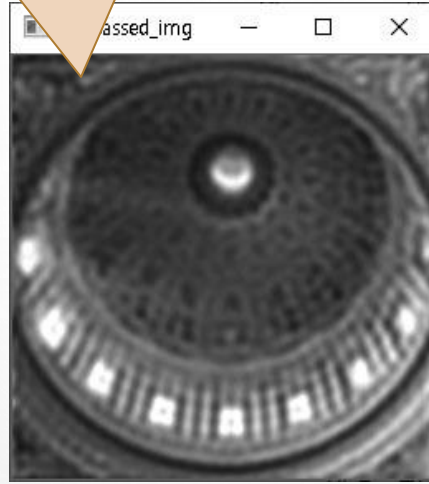
9.4.2 저주파 및 고주파 통과 필터링

◆ 실행결과

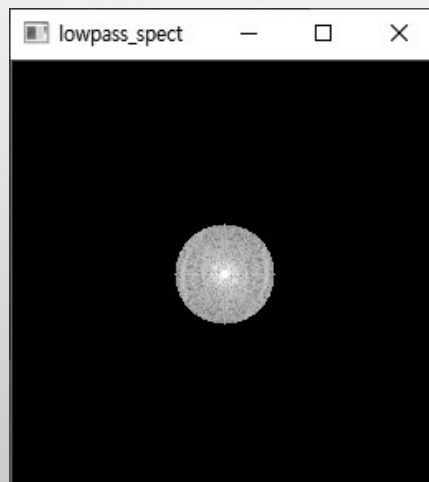
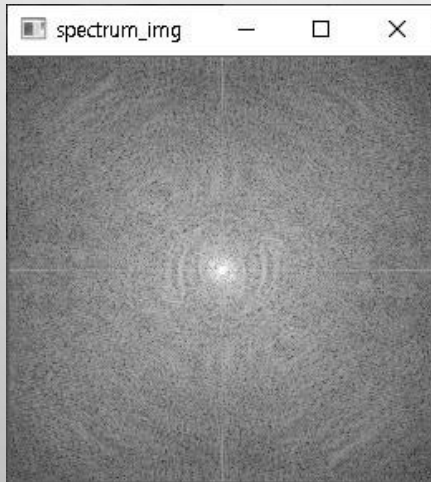
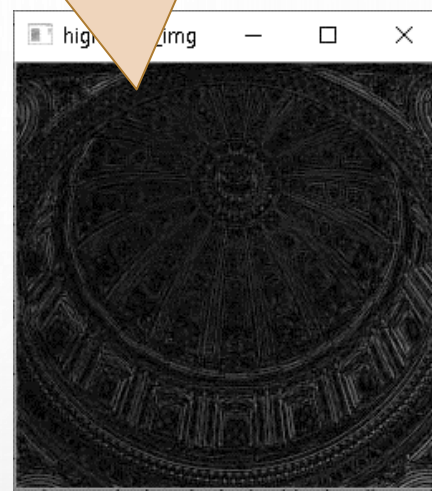


입력영상

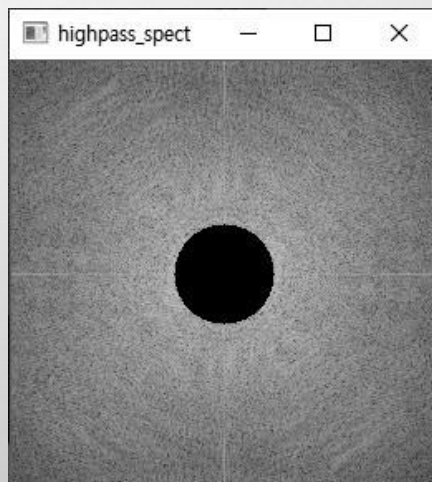
저주파 통과결과 - 흐림 현상



고주파 통과 결과 - 에지 영상



저주파 통과 후
주파수 스펙트럼 영상



고주파 통과 후
주파수 스펙트럼 영상

9.4.3 버터워스, 가우시안 필터링

◆대역 통과 필터

- ❖ 특정한 대역에서 급격하게 값을 제거하기 때문에 결과 영상의 화질 저하
- ❖ 객체의 경계부분 주위로 잔물결 같은 무늬(ringing pattern) 나타남

◆해결방법

- ❖ 필터 원소값을 차단 주파수에서 급격하게 0으로하지 않고 완만한 경사 이루도록 구성
- ❖ 버터워즈 필터(Butterworth filter)나 가우시안 필터(Gaussian filter)

9.4.3 버터워스, 가우시안 필터링

◆가우시안 필터

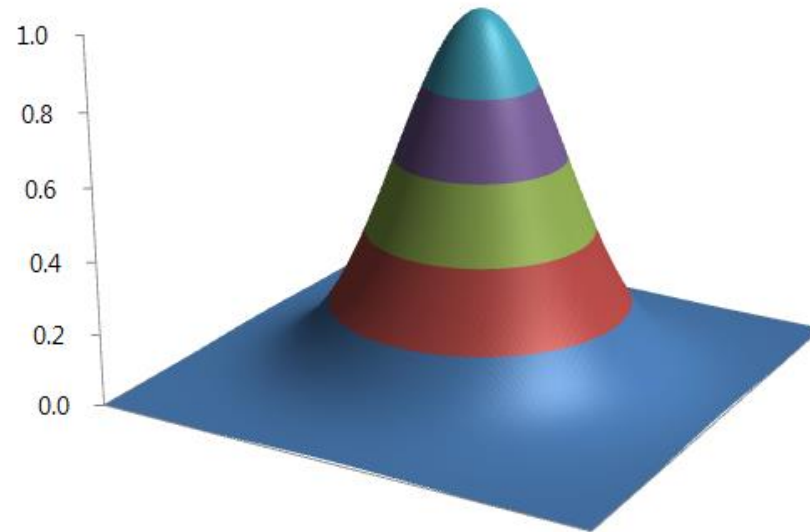
❖ 필터 원소의 구성을 가우시안 함수의 수식 분포를 갖게 함으로써 차단 주파수 부분을 점진적으로 구성한 것

$$f(x, y) = \exp\left(-\frac{dx^2 + dy^2}{2R^2}\right),$$

$dx = x - \text{center}.x$
 $dy = y - \text{center}.y$
 R : 주파수 차단 반지름



필터 계수를 밝기로 표현



필터 계수를 3차원 표현

9.4.3 버터워스, 가우시안 필터링

◆버터워즈 필터

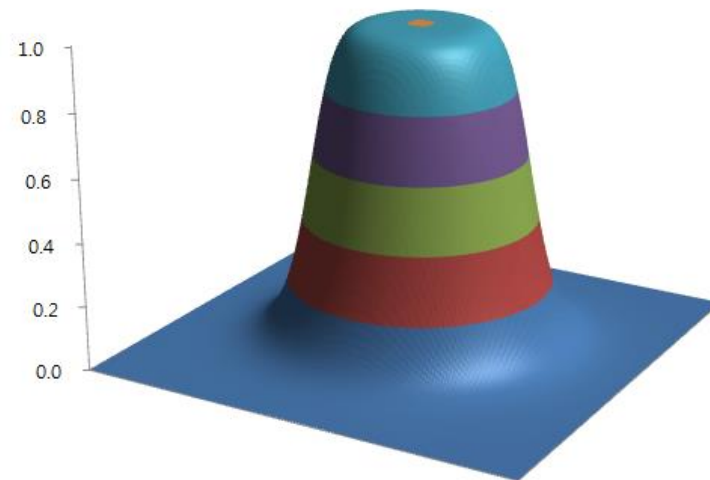
❖ 차단 주파수 반지름 위치(R)와 지수의 승수인 n 값에 따라서 차단 필터의 반지름과 포물선의 곡률 결정

$$f(x, y) = -\frac{1}{1 + \left(\frac{\sqrt{dx^2 + dy^2}}{R}\right)^{2n}},$$

$dx = x - \text{center}.x$
 $dy = y - \text{center}.y$
 R : 주파수 차단 반지름



필터 계수를 밝기로 표현



필터 계수를 3차원 표현

9.4.3 버터워스, 가우시안 필터링

예제 9.4.2

주파수 영역 필터링2 - FFT_filtering2.py

```
01 import numpy as np, cv2
02 from Common.fft2d import FFT, IFFT, calc_spectrum    # 2차원 푸리에 변환 함수 импорт
03 import matplotlib.pyplot as plt                    # 그래프 그리기 импорт
04 from mpl_toolkits.mplot3d import Axes3D            # 3차원 그래프 라이브러리 импорт
05
06 def get_gaussianFilter(shape, R):                  # 가우시안 필터 생성 함수
07     u = np.array(shape)//2
08     y = np.arange(-u[0], u[0], 1)                # x축 범위 및 간격 지정
09     x = np.arange(-u[1], u[1], 1)                # y축 범위 및 간격 지정
10     x, y = np.meshgrid(x, y)                     # x, y 좌표 정방행렬 생성
11     filter = np.exp(-(x**2 + y**2)/ (2 * R**2))
12     return x, y, filter if len(shape) < 3 else cv2.merge([filter, filter])
13
14 def get_butterworthFilter(shape, R, n):            # 버터워스 필터 생성 함수
15     u = np.array(shape)//2
16     y = np.arange(-u[0], u[0], 1)
17     x = np.arange(-u[1], u[1], 1)
18     x, y = np.meshgrid(x, y)
19     dist = np.sqrt(x**2 + y**2)
20     filter = 1 / (1 + np.power(dist / R, 2 * n))
21     return x, y, filter if len(shape) < 3 else cv2.merge([filter, filter])
```

OpenCV 함수는
필터 2채널로 구성

9.4.3 버터워스, 가우시안 필터링

```
23 image = cv2.imread("images/filter1.jpg", cv2.IMREAD_GRAYSCALE)
24 if image is None: raise Exception("영상파일 읽기 에러")
25
26 mode = 2
27 dft, spectrum = FFT(image, mode) # FFT 수행 및 셔플링
28 x1, y1, gauss_filter = get_gaussianFilter(dft.shape, 30) # 필터 생성
29 x2, y2, butter_filter = get_butterworthFilter(dft.shape, 30, 10)
30
31 filtered_dft1 = dft * gauss_filter # 주파수 공간 필터링- 원소 간 곱셈
32 filtered_dft2 = dft * butter_filter
33 gauss_img = IFFT(filtered_dft1, image.shape, mode) # 역푸리에 변환
34 butter_img= IFFT(filtered_dft2, image.shape, mode)
35 spectrum1= calc_spectrum(filtered_dft1) # 필터링 후, 주파수 스펙트럼 영상
36 spectrum2= calc_spectrum(filtered_dft2)
37
```

가우시안 마스크
생성위한 표준편차

차단 반지름 및 곡률

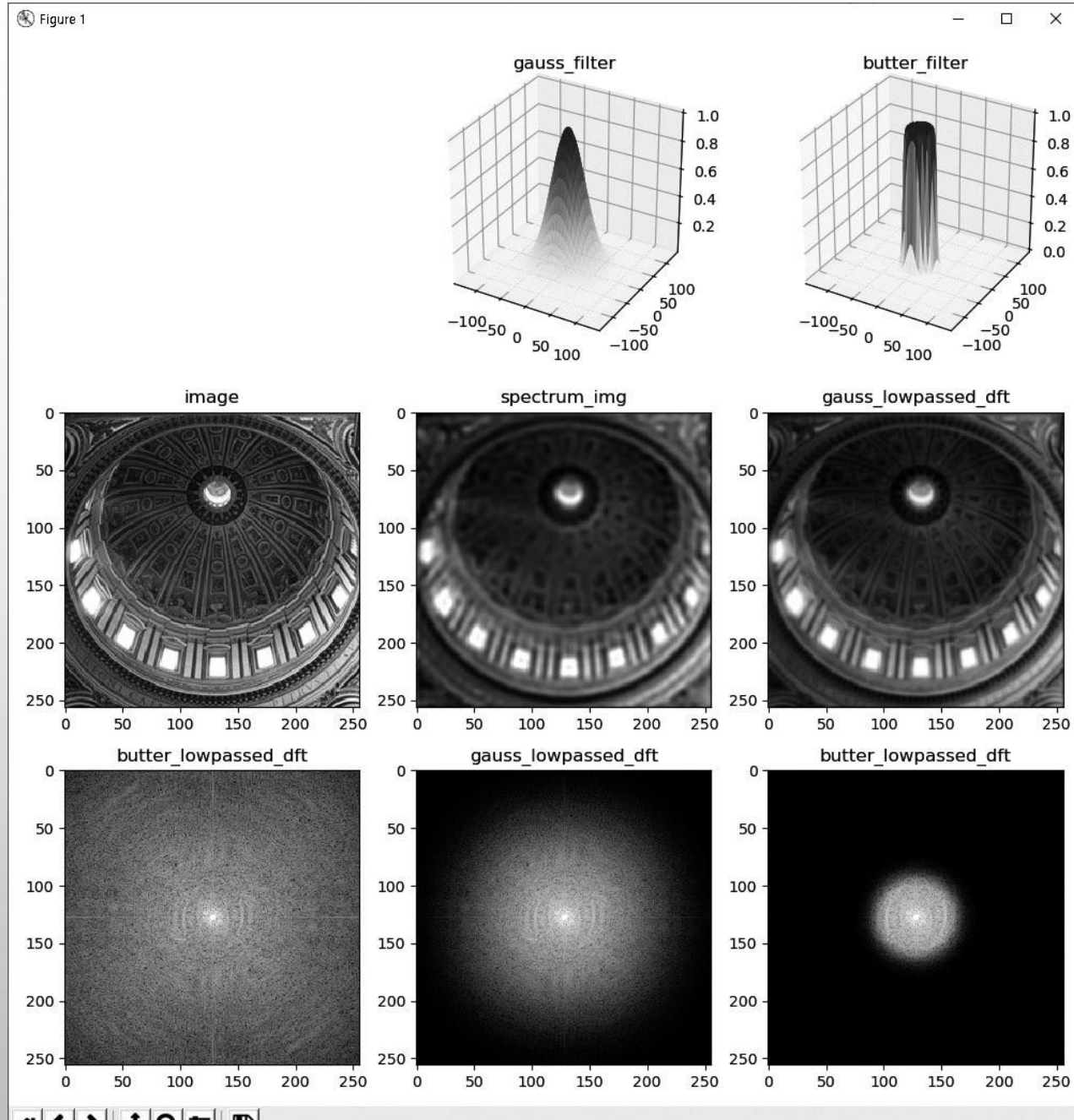
9.4.3 버터워스, 가우시안 필터링

```
38 if mode==3:                                     # OpenCV 함수는 2채널 사용하기에
39     gauss_filter, butter_filter = gauss_filter[:, :, 0], butter_filter[:, :, 0]
40
41 plt.figure(figsize=(10,10))                       # 그래프 생성
42 ax1 = plt.subplot(332, projection='3d')            # 3차원 그래프
43 ax1.plot_surface(x1, y1, gauss_filter, cmap='RdPu'), plt.title("gauss_filter")
44 ax2 = plt.subplot(333, projection='3d')
45 ax2.plot_surface(x2, y2, butter_filter, cmap='RdPu'), plt.title("butter_filter")
46
47 titles = ["input image", "butter_lowpassed_image", "gauss_lowpassed_image",
48           "input spectrum", "gauss_lowpassed_spectrum", "butter_lowpassed_spectrum"]
49 images = [image, butter_img, gauss_img, spectrum, spectrum1, spectrum2] # 결과 영상 행렬
50 plt.gray()                                       # 명암도 영상으로 표시
51 for i, t in enumerate(titles):
52     plt.subplot(3,3,i+4), plt.imshow(images[i]), plt.title(t)
53 plt.tight_layout(), plt.show()
```

한 채널만 그래프로 그림

9.4.3 버터워스, 가우시안 필터링

◆ 실행결과



심화예제 – 모아레 제거

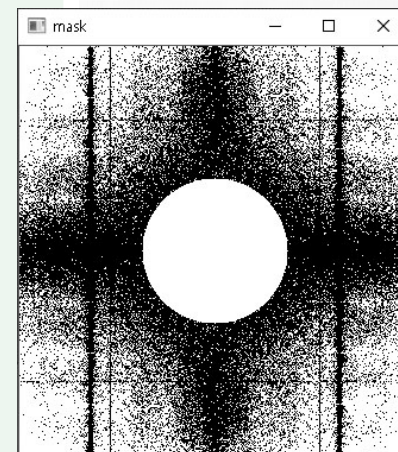
심화예제 9.4.3

모아레 제거 – 08.remove_moire.py

```
01 import numpy as np, cv2
02 from Common.fft2d import FFT, IFFT, calc_spectrum    # 고속 푸리에 변환 관련 함수 импорт
03
04 def onRemoveMoire(val):
05     radius = cv2.getTrackbarPos("radius", title)    # 트랙바 위치 값
06     th = cv2.getTrackbarPos("threshold", title)
07
08     mask= cv2.threshold(spectrum_img, th, 255, cv2.THRESH_BINARY_INV)[1]
09     y, x = np.divmod(mask.shape, 2)[0]              # 마스크 중심 좌표
10     cv2.circle(mask, (x, y), radius, 255, -1)        # 마스크 중심에 흰색 원 그림
11
12     if dft.ndim<3:
13         remv_dft = np.zeros(dft.shape, np.complex)
14         remv_dft.imag = cv2.copyTo(dft.imag, mask=mask)    # 허수부 복사
15         remv_dft.real = cv2.copyTo(dft.real, mask=mask)    # 실수부 복사
16     else:
17         remv_dft = cv2.copyTo(dft, mask=mask)
18
19     result[:, image.shape[1]:] = IFFT(remv_dft, image.shape, mode)    # 관심 영역에 저장
20     cv2.imshow(title, calc_spectrum(remv_dft))    # 마스크된 스펙트럼
21     cv2.imshow("result", result)
22
```

마스크 영상에
원을 그림

OpenCV 함수



심화예제- 모아레 제거

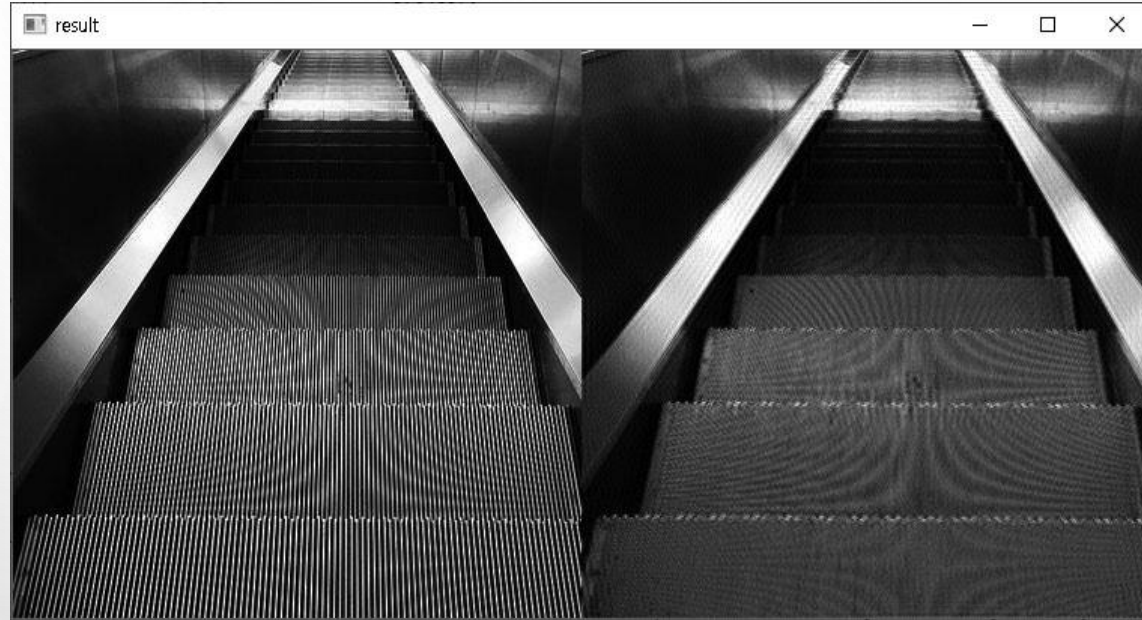
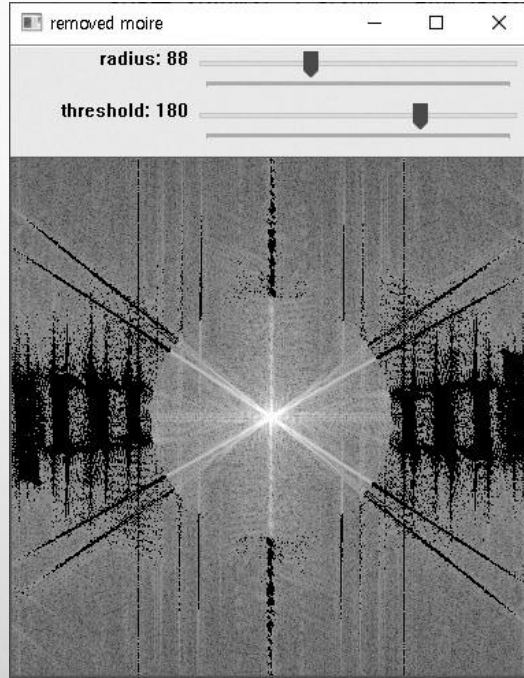
```
23 image = cv2.imread("images/mo2.jpg", cv2.IMREAD_GRAYSCALE)
24 if image is None: raise Exception("영상파일 읽기 에러")
25
26 mode = 3
27 result = cv2.repeat(image, 1, 2)
28 dft, spectrum_img = FFT(image, mode)
29
30 title = "removed moire"
31 cv2.imshow("result", result)
32 cv2.imshow(title, spectrum_img)
33 cv2.createTrackbar("radius", title, 10, 255, onRemoveMoire)
34 cv2.createTrackbar("threshold", title, 120, 255, onRemoveMoire)
35 cv2.waitKey(0)
```

디폴트로 OpenCV dft() 함수 수행

FFT 방법 선택
원본 영상+결과 영상
OpenCV dft() 함수 수행

심화예제- 모아레 제거

◆ 실행결과



9.5 이산 코사인 변환



9.5 이산 코사인 변환

◆ 이산 코사인 변환(DCT: Discrete Cosine Transform)

- ❖ 1974년 미국의 텍사스 대학(University of Texas)에서 라오 교수 팀이 발표한 직교변환에 관한 논문
 - 영상 신호의 에너지 집중 특성이 뛰어나서 영상 압축에 효과적인 주파수 변환 방법을 찾던 중
- ❖ 이산 푸리에 변환(DFT)에서 실수부만 취하고, 허수부분 제외

주파수 영역 신호

$$F(k) = C(k) \cdot \sum_{n=0}^{N-1} g[n] \cdot \cos\left(\frac{(2n+1)k\pi}{2N}\right)$$

$$g[n] = \sum_{k=0}^{N-1} C(k) \cdot F(k) \cdot \cos\left(\frac{(2n+1)k\pi}{2N}\right)$$

공간 영역 신호

$$\text{단, } k=0, \dots, N-1, \quad C(k) = \begin{cases} \sqrt{\frac{1}{N}}, & \text{if } k=0 \\ \sqrt{\frac{2}{N}}, & \text{if } k \neq 0 \end{cases}$$

9.5 이산 코사인 변환

◆ 2차원 DCT

$$F(k, l) = C(k) \cdot C(l) \cdot \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} g[n, m] \cdot \cos\left(\frac{(2n+1)k\pi}{2N}\right) \cdot \cos\left(\frac{(2m+1)l\pi}{2M}\right)$$

$$g[n, m] = \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} C(k) \cdot C(l) \cdot F(k, l) \cdot \cos\left(\frac{(2n+1)k\pi}{2N}\right) \cdot \cos\left(\frac{(2m+1)l\pi}{2M}\right)$$

$$\text{단, } k=0, \dots, N-1, \quad C(k) = \begin{cases} \sqrt{\frac{1}{N}}, & \text{if } k=0 \\ \sqrt{\frac{2}{N}}, & \text{if } k \neq 0 \end{cases}$$

$$l=0, \dots, M-1, \quad C(l) = \begin{cases} \sqrt{\frac{1}{M}}, & \text{if } l=0 \\ \sqrt{\frac{2}{M}}, & \text{if } l \neq 0 \end{cases}$$

❖ 전체 영상을 한 번에 변환시키는 것이 아니라 영상을 작은 블록으로 나누어서 수행

- 블록의 크기를 키울수록 압축의 효율이 높아지지만, 변환의 구현이 어려워지고 속도
- 일반적으로 8×8 크기 사용

9.5 이산 코사인 변환

예제 9.5.1

이산 코사인 변환 - 09.dct.py

```
01 import numpy as np, cv2, math
02 import scipy.fftpack as sf                # scipy 모듈의 fftpack 클래스 임포트
03
04 def cos(n, k, N):                          # 코사인 함수
05     return math.cos((n+1/2) * math.pi * k/N)
06
07 def C(k, N):                              # 상수 계산 함수
08     return math.sqrt(1/N) if k==0 else math.sqrt(2/N)
09
10 def dct(g):                               # 1차원 dct 수행 함수
11     N = len(g)
12     f = [C(k, N) * sum(g[n] * cos(n, k, N) for n in range(N)) for k in range(N)]
13     return np.array(f, np.float32)
14
15 def idct(f):                              # 1차원 역dct 수행 함수
16     N = len(f)
17     g = [sum(C(k, N) * f[k] * cos(n, k, N) for k in range(N)) for n in range(N)]
18     return np.array(g)
19
```

리스트를 ndarray로 변환후 반환

9.5 이산 코사인 변환

저자 구현 함수

```
20 def dct2(image): # 2차원 dct 수행 함수
21     tmp = [dct(row) for row in image]
22     dst = [dct(row) for row in np.transpose(tmp)]
23     return np.transpose(dst) # 전치 환원 후 반환
24
25 def idct2(image): # 2차원 역dct 수행 함수
26     tmp = [idct(row) for row in image]
27     dst = [idct(row) for row in np.transpose(tmp)]
28     return np.transpose(dst) # 전치 환원 후 반환
29
30 def scipy_dct2(a): # scipy 모듈 이용한 2차원 dct
31     tmp = sf.dct(a, axis=0, norm='ortho' ) # 세로 방향 1차원 dct
32     return sf.dct(tmp, axis=1, norm='ortho' ) # 가로 방향 1차원 dct
33
34 def scipy_idct2(a):
35     tmp = sf.idct(a, axis=0, norm='ortho') # 세로 방향 1차원 idct
36     return sf.idct(tmp, axis=1, norm='ortho') # 가로 방향 1차원 idct
37
```

scipy 모듈 제공 함수

9.5 이산 코사인 변환

```
38 block = np.zeros((3,5), np.uint8)
39 cv2.randn(block, 128, 50) # 랜덤 원소 생성
40
41 dct1 = dct2(block) # 2차원 dct 수행
42 dct2 = scipy_dct2(block) # scipy 1차원 dct 두 번 수행
43 dct3 = sf.dctn(block, shape=block.shape, norm='ortho') # scipy 2차원 dct 한번만 수행
44 dct4 = cv2.dct(block.astype('float32')) # OpenCV 함수
45
46 idct1 = idct2(dct1) # 2차원 idct 수행
47 idct2 = scipy_idct2(dct2)
48 idct4 = sf.idctn(dct4, shape=dct4.shape, norm='ortho') # 2차원dct 수행 다른 방식
49 idct3 = cv2.dct(dct3, flags=cv2.DCT_INVERSE) # OpenCV 함수
50
51 print('block=', block)
52 print('dct1(저자구현 함수)=\n', dct1)
53 print('dct2(scipy 모듈 함수1)=\n', dct2)
54 print('dct3(scipy 모듈 함수2)=\n', dct3)
55 print('dct4(OpenCV 함수)=\n', dct4)
56 print()
57 print('idct1(저자구현 함수)=\n', cv2.convertScaleAbs(idct1))
58 print('idct2(scipy 모듈 함수1)=\n', cv2.convertScaleAbs(idct2))
59 print('idct3(scipy 모듈 함수2)=', cv2.convertScaleAbs(idct3))
60 print('idct4(OpenCV 함수)=', cv2.convertScaleAbs(idct4))
```


9.5 이산 코사인 변환

◆ 실행결과

(0, 0) 위치 DC 계수
- 공간 영역의 화소값의
평균에 해당하는 값으로
서 에너지가 집중

나머지 계수 -
AC 계수들

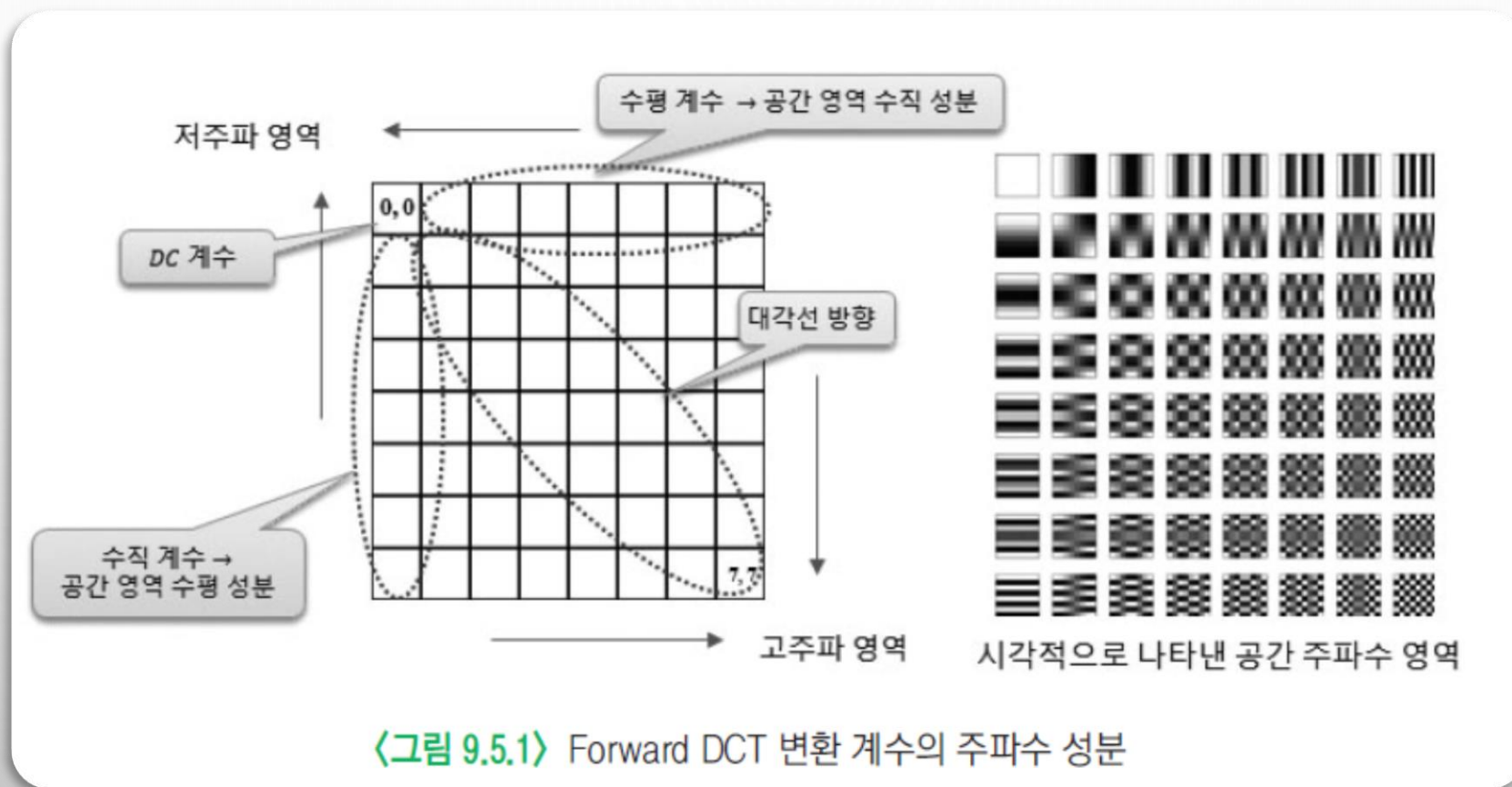
```
Run: 09.dct x
C:\Python\python.exe D:/source/chap09/09.dct.py
block=
[[128 136 93 107 190]
 [114 96 143 154 71]
 [166 126 193 30 33]]
dct1(저자구현 함수)=
[[ 459.59402  53.96968 -22.50121  1.200144  44.138367]
 [ 33.52015 -100.550446  75.75286 -22.76198 -59.752857]
 [ 8.398399  32.89206  38.765003 -61.45686  32.826427]]
dct2(scipy 모듈 함수1)=
[[ 459.59402375  53.96967963 -22.50120922  1.2001449  44.13836861]
 [ 33.5201432 -100.55044607  75.75285972 -22.76198135 -59.75285972]
 [ 8.39841255  32.89205934  38.76500392 -61.45686101  32.82642946]]
dct3(scipy 모듈 함수2)=
[[ 459.59402375  53.96967963 -22.50120922  1.2001449  44.13836861]
 [ 33.5201432 -100.55044607  75.75285972 -22.76198135 -59.75285972]
 [ 8.39841255  32.89205934  38.76500392 -61.45686101  32.82642946]]
dct4(OpenCV 함수)=
[[ 459.594  53.969677 -22.501207  1.200143  44.138374]
 [ 33.520145 -100.550446  75.752846 -22.76198 -59.752853]
 [ 8.398418  32.892063  38.765 -61.456863  32.826435]]

idct1(저자구현 함수)=
[[128 136 93 107 190]
 [114 96 143 154 71]
 [166 126 193 30 33]]
idct2(scipy 모듈 함수1)=
[[128 136 93 107 190]
 [114 96 143 154 71]
 [166 126 193 30 33]]
idct3(scipy 모듈 함수2)=
[[128 136 93 107 190]
 [114 96 143 154 71]
 [166 126 193 30 33]]
idct4(OpenCV 함수)=
[[128 136 93 107 190]
 [114 96 143 154 71]
 [166 126 193 30 33]]
```

9.5 이산 코사인 변환

◆DCT 계수의 주파수 특성

- ❖ 왼쪽 상단으로 갈수록 저주파 영역이며, 오른쪽 하단으로 갈수록 고주파 영역



9.5 이산 코사인 변환

◆DCT 주파수 영역 필터 구성

1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

DC Pass

	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1

High Pass

1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Low Pass

0	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Vertical Pass

0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0

Horizontal Pass

9.5 이산 코사인 변환

예제 9.5.2

DCT를 이용한 주파수 영역 필터링 - 10.DCT_filtering.py

```
01 import numpy as np, cv2
02 from Common.dct2d import dct2, idct2, scipy_dct2, scipy_idct2 # dct 관련 함수 импорт
03
04 def dct2_mode(block, mode):                                # 2차원 dct 함수 - 방식 선택
05     if mode==1: return dct2(block)
06     elif mode==2: return scipy_dct2(block)
07     elif mode==3: return cv2.dct(block.astype('float32'))
08
09 def idct2_mode(block, mode):                                # 2차원 idct 함수 - 방식 선택
10     if mode==1: return idct2(block)
11     elif mode==2: return scipy_idct2(block)
12     elif mode==3: return cv2.dct(block, flags=cv2.DCT_INVERSE)
13
14 def dct_filtering(img, filter, M, N):                        # 주파수 영역 필터링 함수
15     dst = np.empty(img.shape, np.float32)
16     for i in range(0, img.shape[0], M):                    # 입력 영상 순회
17         for j in range(0, img.shape[1], N):
18             block = img[i:i+M, j:j+N]                      # 블록 참조
19             new_block = dct2_mode(block, mode)              # 블록 DCT 수행
20             new_block = new_block * filter                  # 곱셈을 통한 필터링
21             dst[i:i+M, j:j+N] = idct2_mode(new_block, mode) # 역DCT
22     return cv2.convertScaleAbs(dst)
```

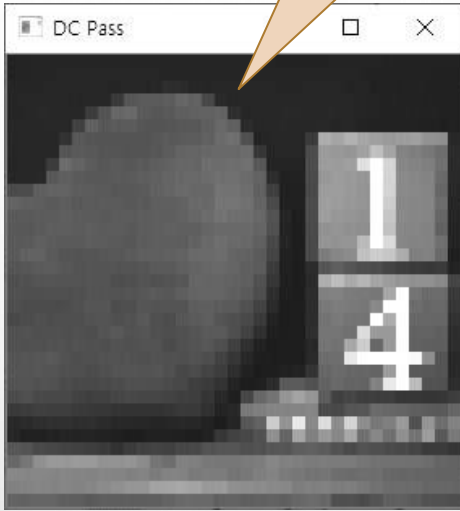

9.5 이산 코사인 변환

```
24 image = cv2.imread("images/dct.jpg", cv2.IMREAD_GRAYSCALE)
25 if image is None: raise Exception("영상파일 읽기 에러")
26
27 mode = 3                                # dct 방식 선택
28 M, N = 8, 8                             # 블록 크기
29 filters = [np.zeros((M, N), np.float32) for i in range(5)] # 5개 필터 생성 및 0으로 초기화
30 titles = ['DC Pass', 'High Pass', 'Low Pass', 'Vertical Pass', 'Horizontal Pass' ]
31
32 filters[0][0, 0] = 1                     # DC 계수만 1 지정 - DC Pass
33 filters[1][:, 0], filters[2][0, 0] = 1, 0 # 모든 계수 1, DC만 0 지정 - High Pass
34 filters[2][:M//2, :N//2] = 1             # 1/4 영역 1 지정 - Low Pass
35 filters[3][0, 1:] = 1                   # 수직 성분 - Vertical Pass
36 filters[4][1:, 0] = 1                   # 수평 성분 - Horizontal Pass
37
38 for filter, title in zip(filters, titles):
39     dst = dct_filtering(image, filter, M, N)
40     cv2.imshow(title, dst)
41 cv2.imshow("image", image)
42 cv2.waitKey(0)
```

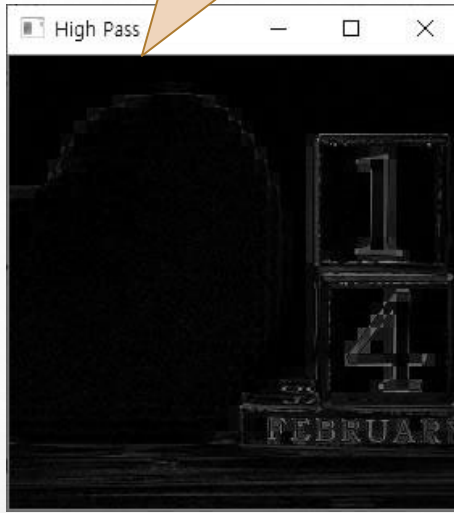

9.5 이산 코사인 변환

◆ 실행결과

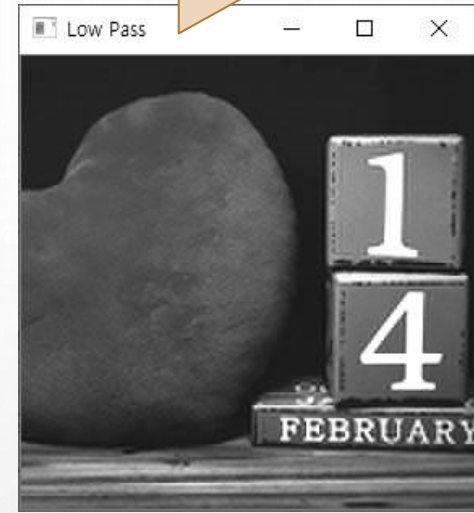
DC 계수만 통과



DC 계수만 0



저주파 통과 필터 -
저주파 영역 16개 계수만 통과



수직방향 통과



수평방향 통과



원본영상

